



ÁRVORE BINÁRIA DE BUSCA BALANCEADA 2



Prof. Me. Darcy H.F.R. Moreira



IMPLEMENTAÇÃO AVL

Estrutura da ABB

Modificações nas funções:

struct No → Altura

Criar Nó

Inserção

Remoção

Novas funções:

Cálculo do fb;

Balacear nó; → Rotações

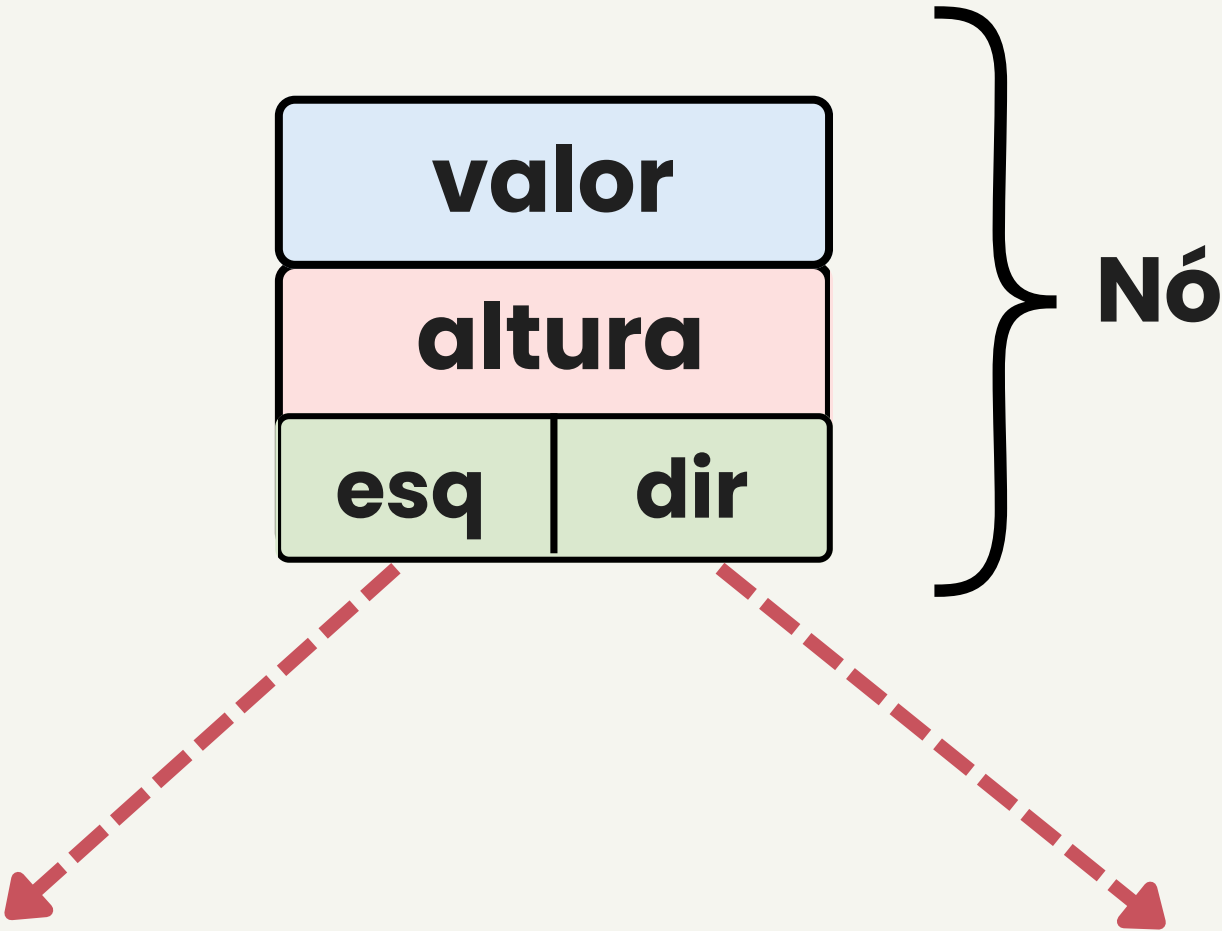
IMPLEMENTAÇÃO AVL

Struct Nó:

- **valor**
- **2 ponteiros** (esquerda e direita)
- **altura**

```

typedef struct NoArv {
    int valor;
    struct NoArv* esq;
    struct NoArv* dir;
    int altura;
} NoArv;
    
```



FUNÇÃO INSERIR COM RECURSÃO

- A função percorre a árvore recursivamente até encontrar um **local vazio (NULL)** onde o valor pode ser **inserido**.
- Quando o local é **encontrado**, um novo nó é **criado e retornado**.
- **Durante o retorno** das chamadas recursivas, cada nó **retorna a si mesmo, atualizando os ponteiros dos filhos** conforme necessário.

```

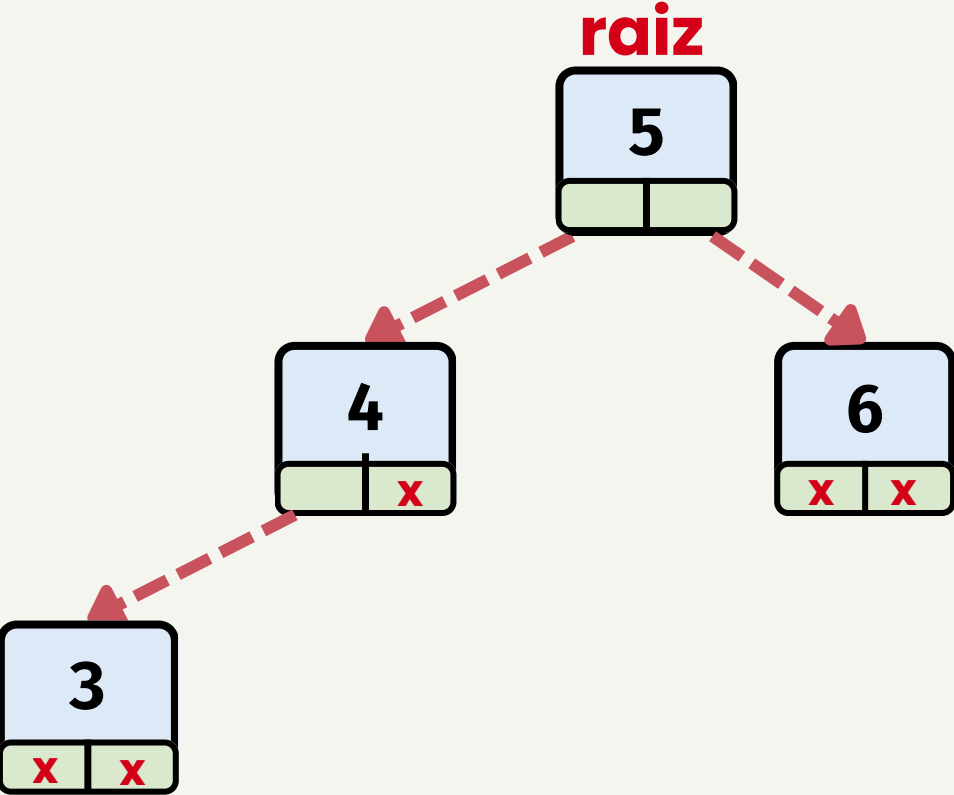
NoArv* inserirRec(NoArv* raiz, int valor) {
    if (raiz == NULL) {
        return criarNo(valor);
    }
    if (valor < raiz->valor) {
        raiz->esq = inserirRec(raiz->esq, valor);
    } else if (valor > raiz->valor) {
        raiz->dir = inserirRec(raiz->dir, valor);
    }
    else {
        printf("Valor %d já existe na árvore. \n", valor);
    }
    return raiz;
}

```

1. **Busca do local da inserção** (Processo Recursivo): Executa a função para esquerda ou direita
2. **Caso base**: Quando for NULL, encontrou → crio o Nó.

FUNÇÃO INSERIR COM RECURSÃO

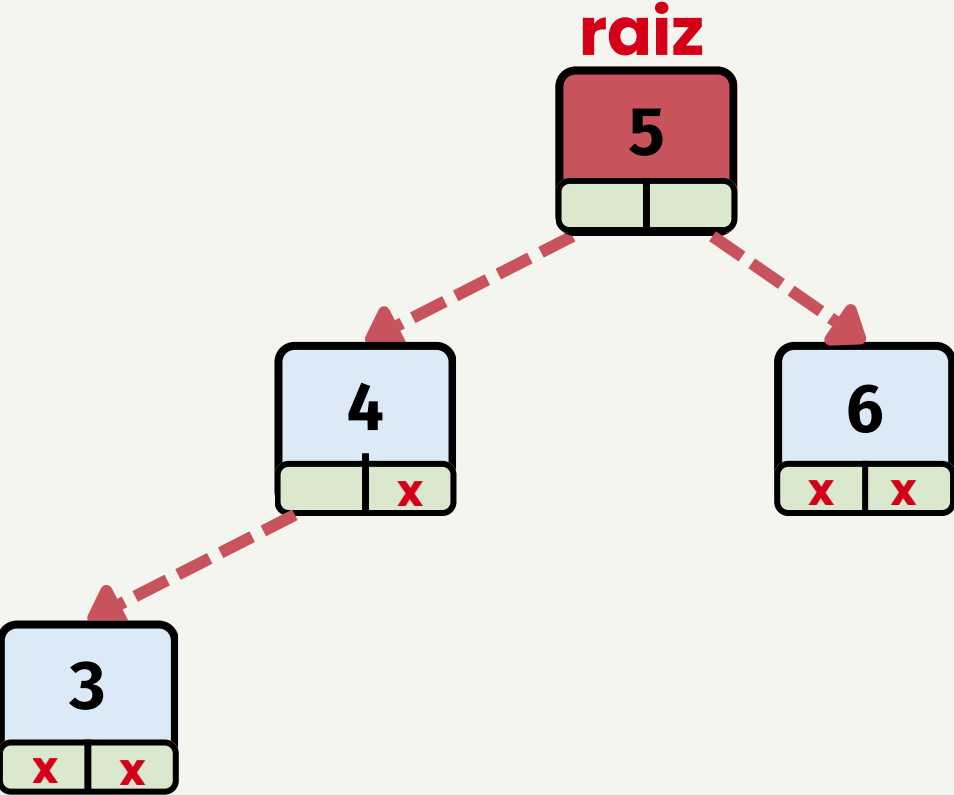
INSERIR: 2



```
NoArv* inserirRec(NoArv* raiz, int valor) {
    if (raiz == NULL) {
        return criarNo(valor);
    }
    if (valor < raiz->valor) {
        raiz->esq = inserirRec(raiz->esq, valor);
    } else if (valor > raiz->valor) {
        raiz->dir = inserirRec(raiz->dir, valor);
    }
    else {
        printf("Valor %d já existe na árvore. \n", valor);
    }
    return raiz;
}
```

FUNÇÃO INSERIR COM RECURSÃO

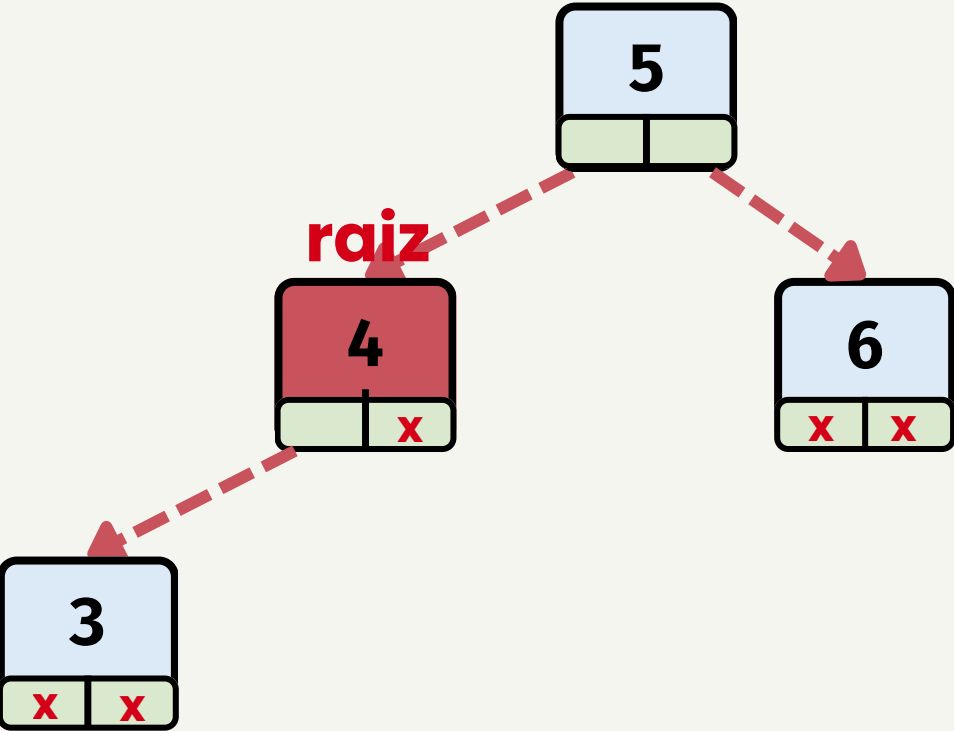
INSERIR: 2



```
NoArv* inserirRec(NoArv* raiz, int valor) {
    if (raiz == NULL) {
        return criarNo(valor);
    }
    if (valor < raiz->valor) {
        raiz->esq = inserirRec(raiz->esq, valor);
    } else if (valor > raiz->valor) {
        raiz->dir = inserirRec(raiz->dir, valor);
    }
    else {
        printf("Valor %d já existe na árvore. \n", valor);
    }
    return raiz;
}
```

FUNÇÃO INSERIR COM RECURSÃO

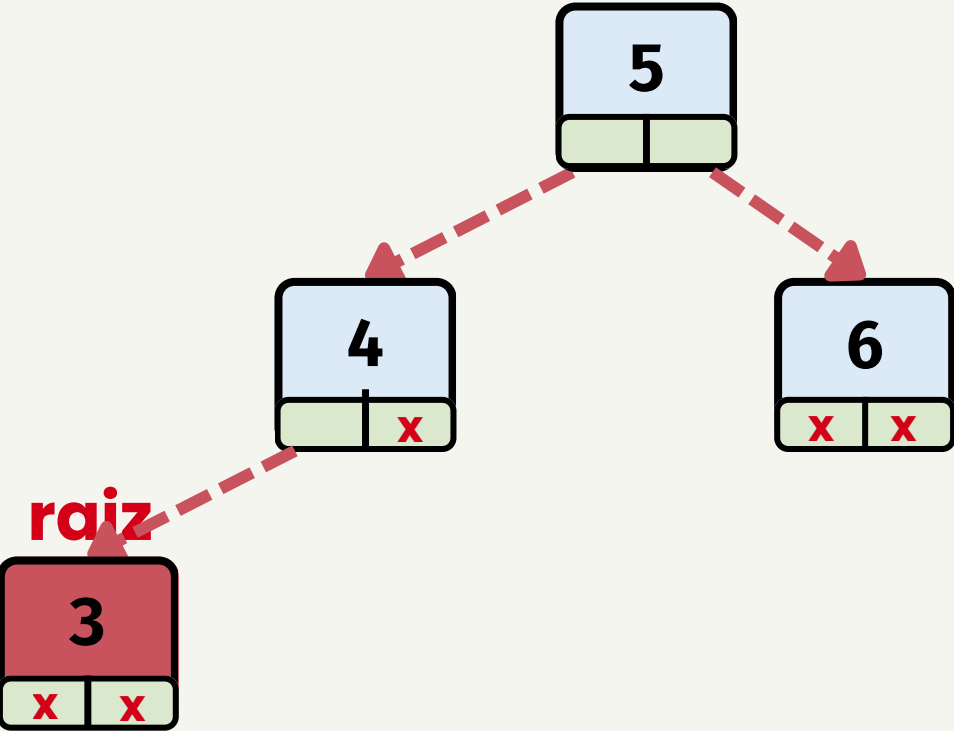
INSERIR: 2



```
NoArv* inserirRec(NoArv* raiz, int valor) {
    if (raiz == NULL) {
        return criarNo(valor);
    }
    if (valor < raiz->valor) {
        raiz->esq = inserirRec(raiz->esq, valor);
    } else if (valor > raiz->valor) {
        raiz->dir = inserirRec(raiz->dir, valor);
    }
    else {
        printf("Valor %d já existe na árvore. \n", valor);
    }
    return raiz;
}
```

FUNÇÃO INSERIR COM RECURSÃO

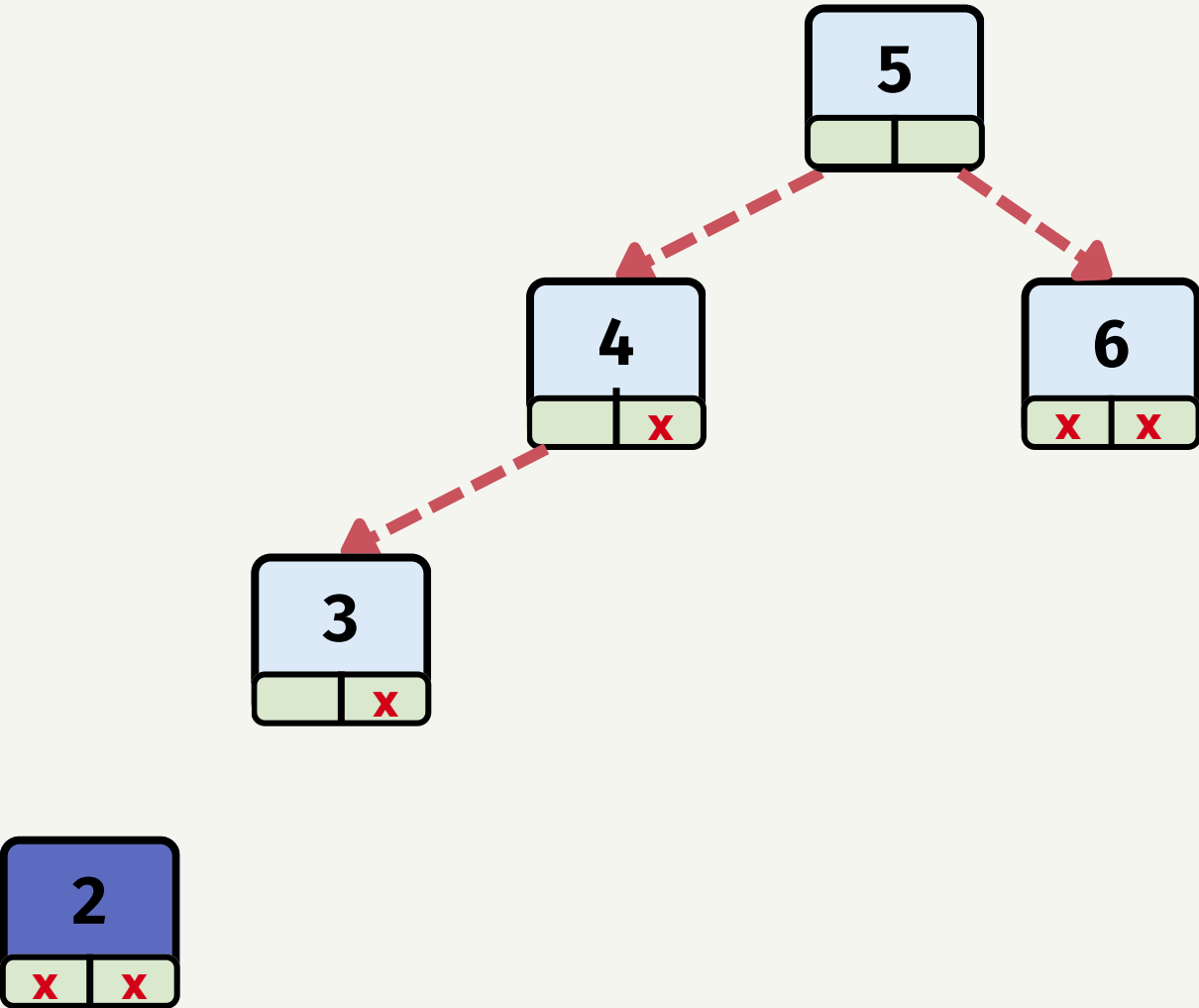
INSERIR: 2



```
NoArv* inserirRec(NoArv* raiz, int valor) {
    if (raiz == NULL) {
        return criarNo(valor);
    }
    if (valor < raiz->valor) {
        raiz->esq = inserirRec(raiz->esq, valor);
    } else if (valor > raiz->valor) {
        raiz->dir = inserirRec(raiz->dir, valor);
    }
    else {
        printf("Valor %d já existe na árvore. \n", valor);
    }
    return raiz;
}
```

FUNÇÃO INSERIR COM RECURSÃO

INSERIR: 2

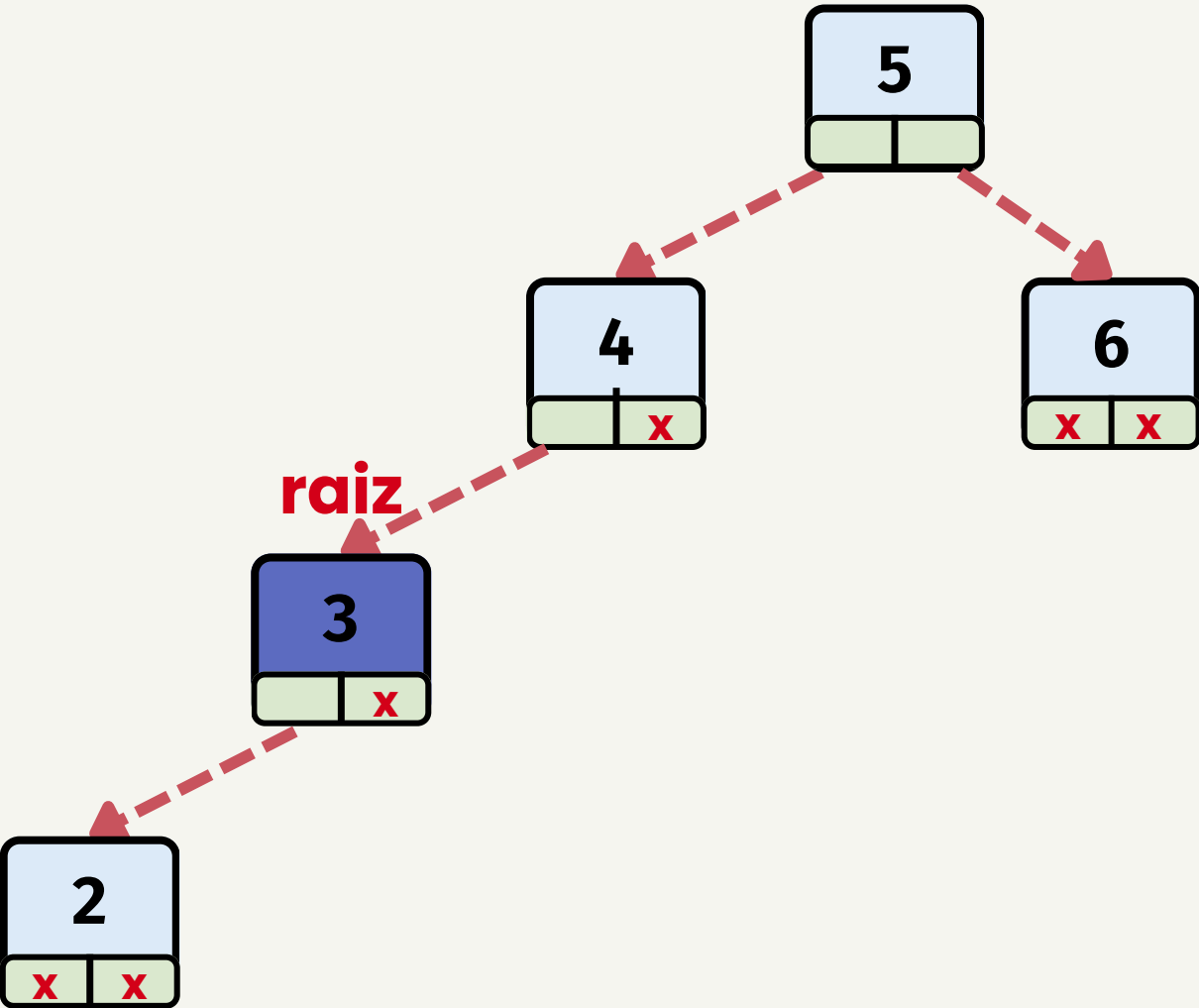


```
NoArv* inserirRec(NoArv* raiz, int valor) {
    if (raiz == NULL) {
        return criarNo(valor);
    }
    if (valor < raiz->valor) {
        raiz->esq = inserirRec(raiz->esq, valor);
    } else if (valor > raiz->valor) {
        raiz->dir = inserirRec(raiz->dir, valor);
    }
    else {
        printf("Valor %d já existe na árvore. \n", valor);
    }
    return raiz;
}
```

Quando o local é encontrado, um novo nó é criado e retornado.

FUNÇÃO INSERIR COM RECURSÃO

INSERIR: 2

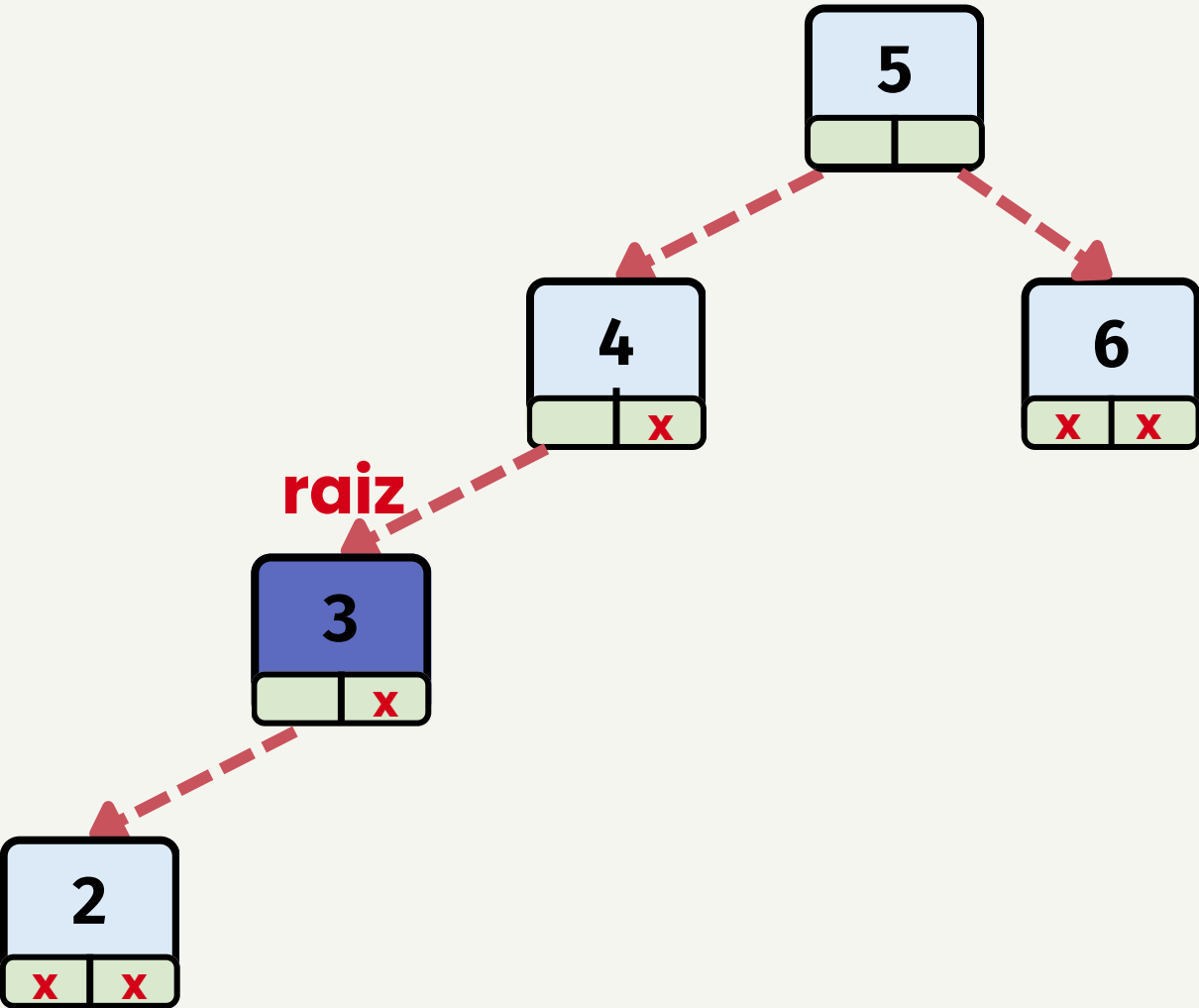


```
NoArv* inserirRec(NoArv* raiz, int valor) {
    if (raiz == NULL) {
        return criarNo(valor);
    }
    if (valor < raiz->valor) {
        raiz->esq = inserirRec(raiz->esq, valor);
    } else if (valor > raiz->valor) {
        raiz->dir = inserirRec(raiz->dir, valor);
    }
    else {
        printf("Valor %d já existe na árvore. \n", valor);
    }
    return raiz;
}
```

Quando o local é encontrado, um novo nó é criado e retornado.

FUNÇÃO INSERIR COM RECURSÃO

INSERIR: 2

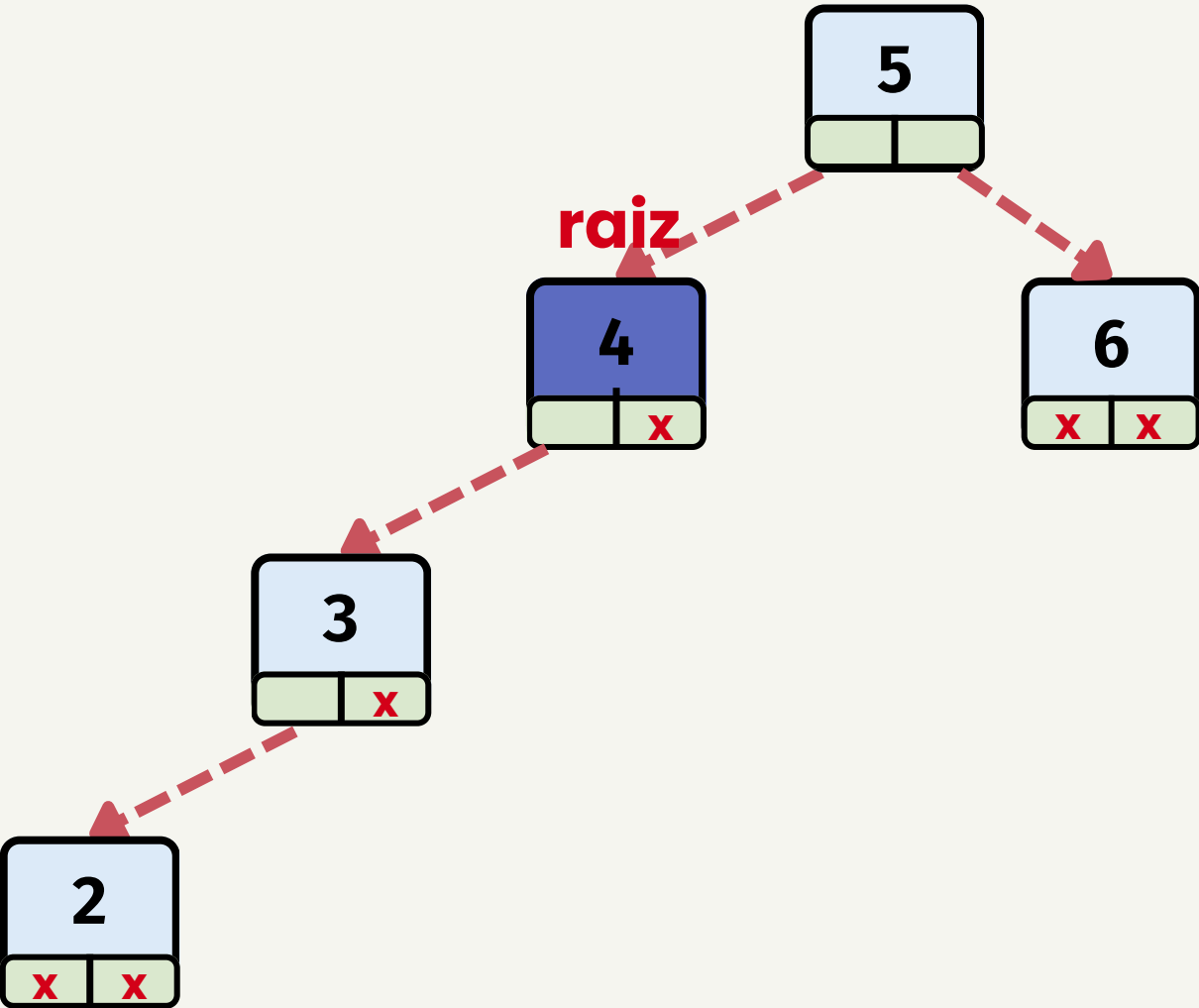


```
NoArv* inserirRec(NoArv* raiz, int valor) {
    if (raiz == NULL) {
        return criarNo(valor);
    }
    if (valor < raiz->valor) {
        raiz->esq = inserirRec(raiz->esq, valor);
    } else if (valor > raiz->valor) {
        raiz->dir = inserirRec(raiz->dir, valor);
    }
    else {
        printf("Valor %d já existe na árvore. \n", valor);
    }
    return raiz;
}
```

Durante o **retorno** das chamadas recursivas, **cada nó retorna a si mesmo**, atualizando os ponteiros dos filhos conforme necessário.

FUNÇÃO INSERIR COM RECURSÃO

INSERIR : 2

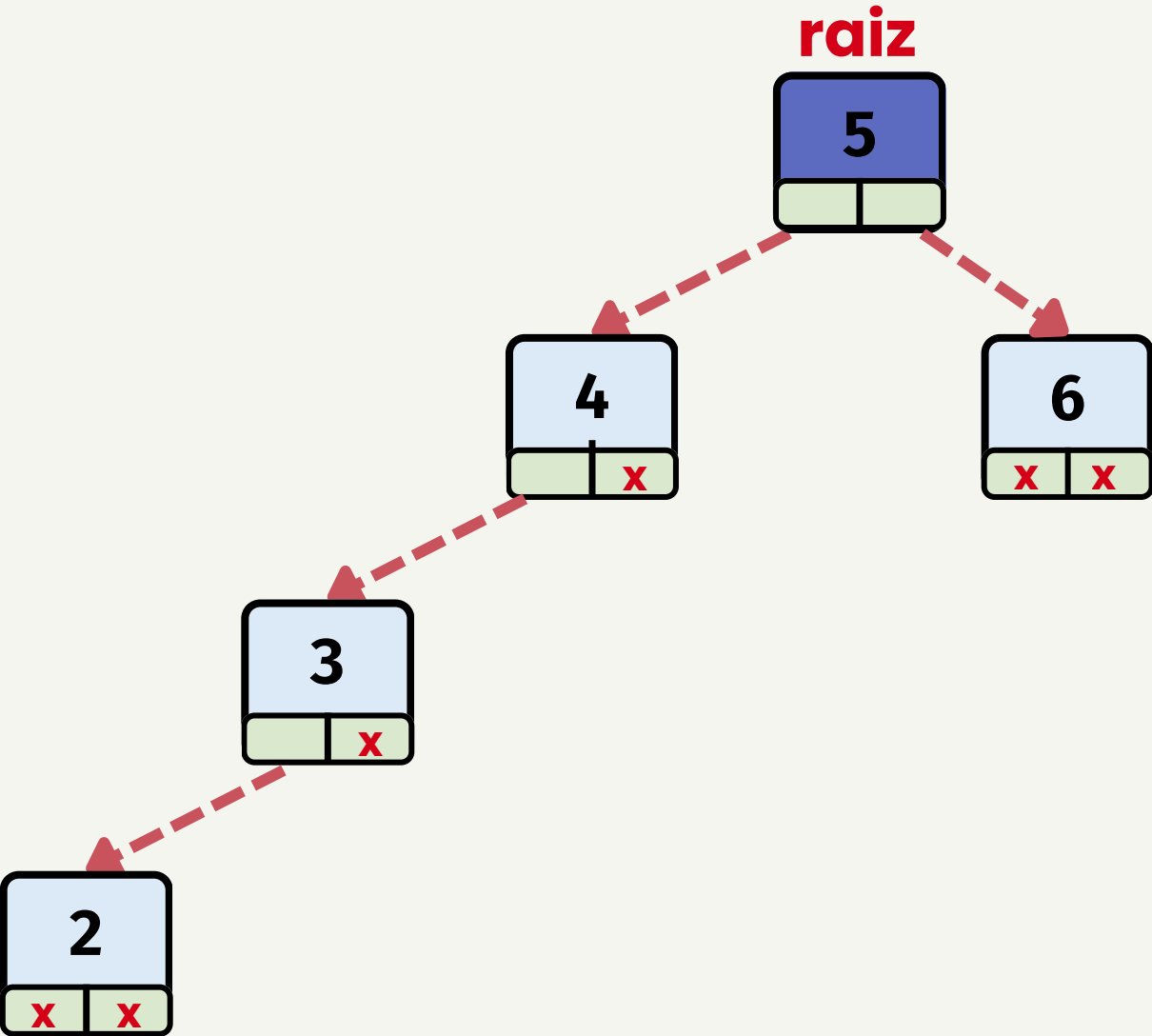


```
NoArv* inserirRec(NoArv* raiz, int valor) {
    if (raiz == NULL) {
        return criarNo(valor);
    }
    if (valor < raiz->valor) {
        raiz->esq = inserirRec(raiz->esq, valor);
    } else if (valor > raiz->valor) {
        raiz->dir = inserirRec(raiz->dir, valor);
    }
    else {
        printf("Valor %d já existe na árvore. \n", valor);
    }
    return raiz;
}
```

Durante o **retorno** das chamadas recursivas, **cada nó retorna a si mesmo**, atualizando os ponteiros dos filhos conforme necessário.

FUNÇÃO INSERIR COM RECURSÃO

INSERIR : 2

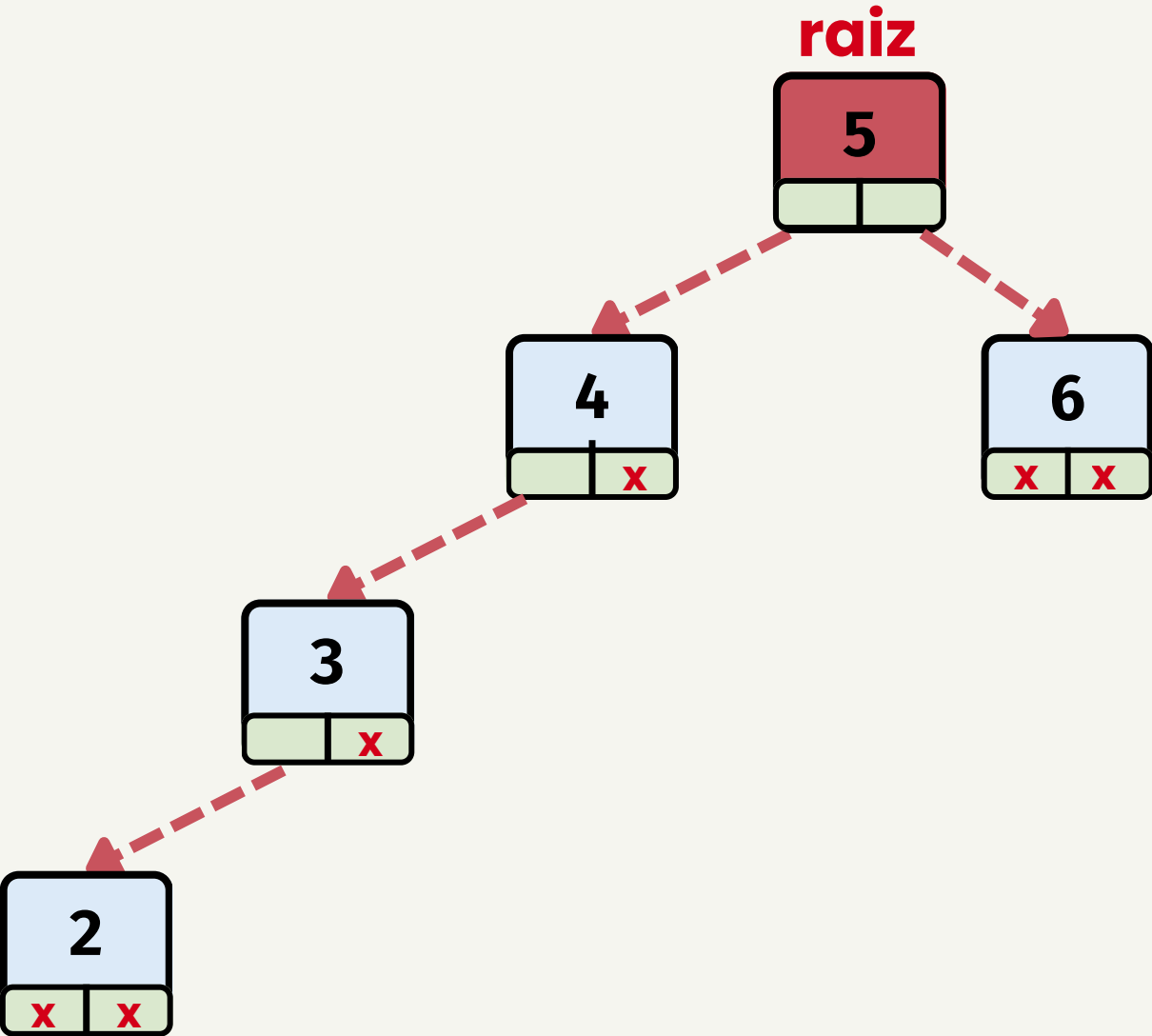


```
NoArv* inserirRec(NoArv* raiz, int valor) {
    if (raiz == NULL) {
        return criarNo(valor);
    }
    if (valor < raiz->valor) {
        raiz->esq = inserirRec(raiz->esq, valor);
    } else if (valor > raiz->valor) {
        raiz->dir = inserirRec(raiz->dir, valor);
    }
    else {
        printf("Valor %d já existe na árvore. \n", valor);
    }
    return raiz;
}
```

Durante o **retorno** das chamadas recursivas, **cada nó retorna a si mesmo**, atualizando os ponteiros dos filhos conforme necessário.

FUNÇÃO INSERIR COM RECURSÃO

INSERIR: 2



```
NoArv* inserirRec(NoArv* raiz, int valor) {
    if (raiz == NULL) {
        return criarNo(valor);
    }
    if (valor < raiz->valor) {
        raiz->esq = inserirRec(raiz->esq, valor);
    } else if (valor > raiz->valor) {
        raiz->dir = inserirRec(raiz->dir, valor);
    }
    else {
        printf("Valor %d já existe na árvore. \n", valor);
    }
    return raiz;
}
```

Durante o **retorno** das chamadas recursivas, **cada nó retorna a si mesmo**, atualizando os ponteiros dos filhos conforme necessário.

FUNÇÃO INSERIR COM RECURSÃO

O que eu preciso alterar para transformar em uma AVL?

INSERIR : 2

```
NoArv* inserirRec(NoArv* raiz, int valor) {  
    if (raiz == NULL) {  
        return criarNo(valor);  
    }  
    if (valor < raiz->valor) {  
        raiz->esq = inserirRec(raiz->esq, valor);  
    } else if (valor > raiz->valor) {  
        raiz->dir = inserirRec(raiz->dir, valor);  
    }  
    else {  
        printf("Valor %d já existe na árvore. \n", valor);  
    }  
    return raiz;  
}
```

FUNÇÃO INSERIR COM RECURSÃO

INSERIR : 2

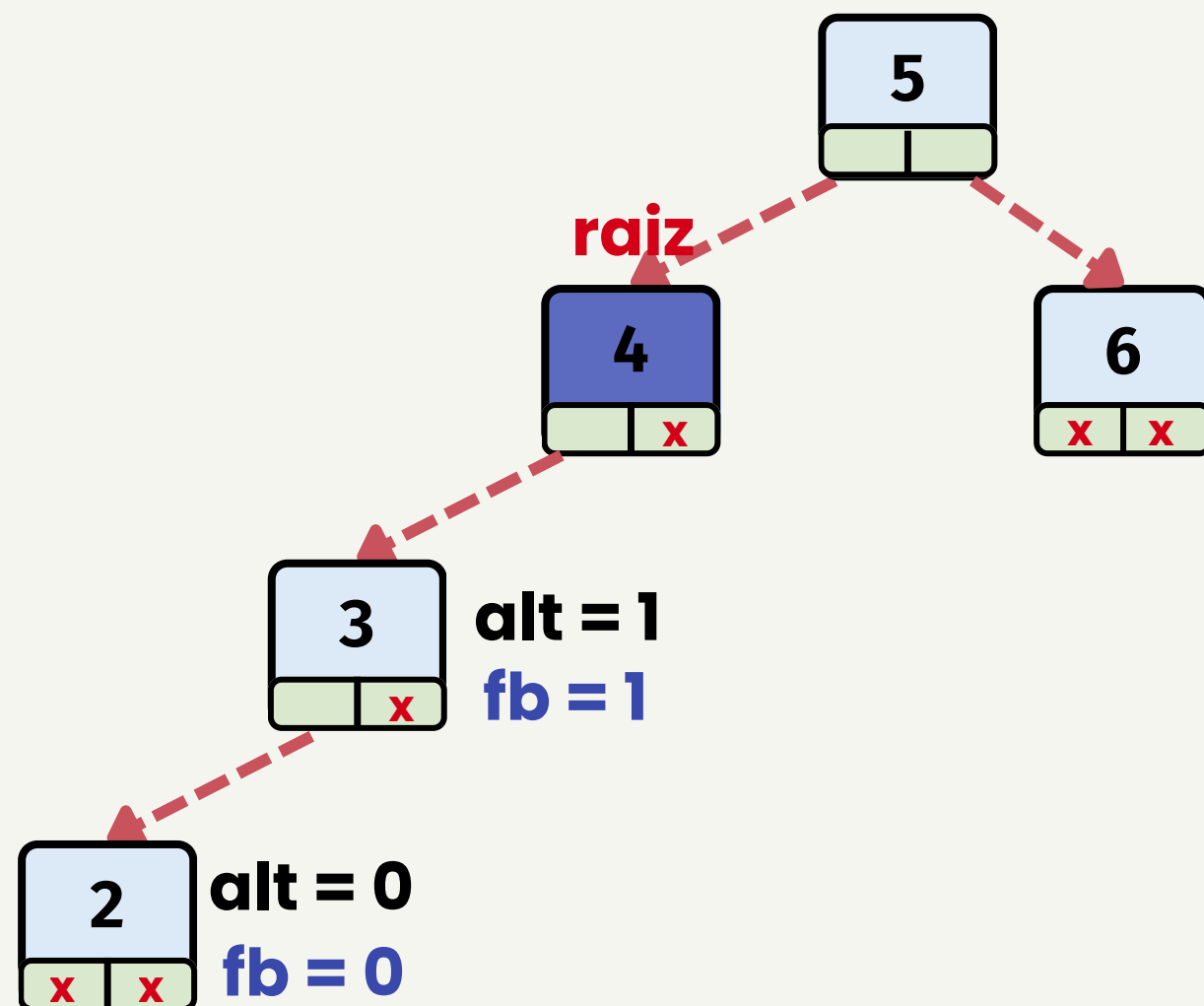
O que eu preciso alterar para transformar em uma AVL?

- Atualizar altura
- Verificar Balaceamento

```
NoArv* inserirRecBalanceada(NoArv* raiz, int valor) {  
    if (raiz == NULL) {  
        return criarNo(valor);  
    }  
    if (valor < raiz->valor) {  
        raiz->esq = inserirRec(raiz->esq, valor);  
    } else if (valor > raiz->valor) {  
        raiz->dir = inserirRec(raiz->dir, valor);  
    }  
    else {  
        printf("Valor %d já existe na árvore. \n", valor);  
    }  
  
    raiz->altura = 1 + maiorValor(alturaDoNo(raiz->esq), alturaDoNo(raiz->dir));  
    raiz = balancear(raiz);  
  
    return raiz;  
}
```

FUNÇÃO INSERIR COM RECURSÃO

INSERIR : 2



```
NoArv* inserirRecBalanceada(NoArv* raiz, int valor) {  
    if (raiz == NULL) {  
        return criarNo(valor);  
    }  
    if (valor < raiz->valor) {  
        raiz->esq = inserirRec(raiz->esq, valor);  
    } else if (valor > raiz->valor) {  
        raiz->dir = inserirRec(raiz->dir, valor);  
    }  
    else {  
        printf("Valor %d já existe na árvore. \n", valor);  
    }  
  
    raiz->altura = 1 + maiorValor(alturaDoNo(raiz->esq), alturaDoNo(raiz->dir));  
    raiz = balancear(raiz);  
  
    return raiz;  
}
```

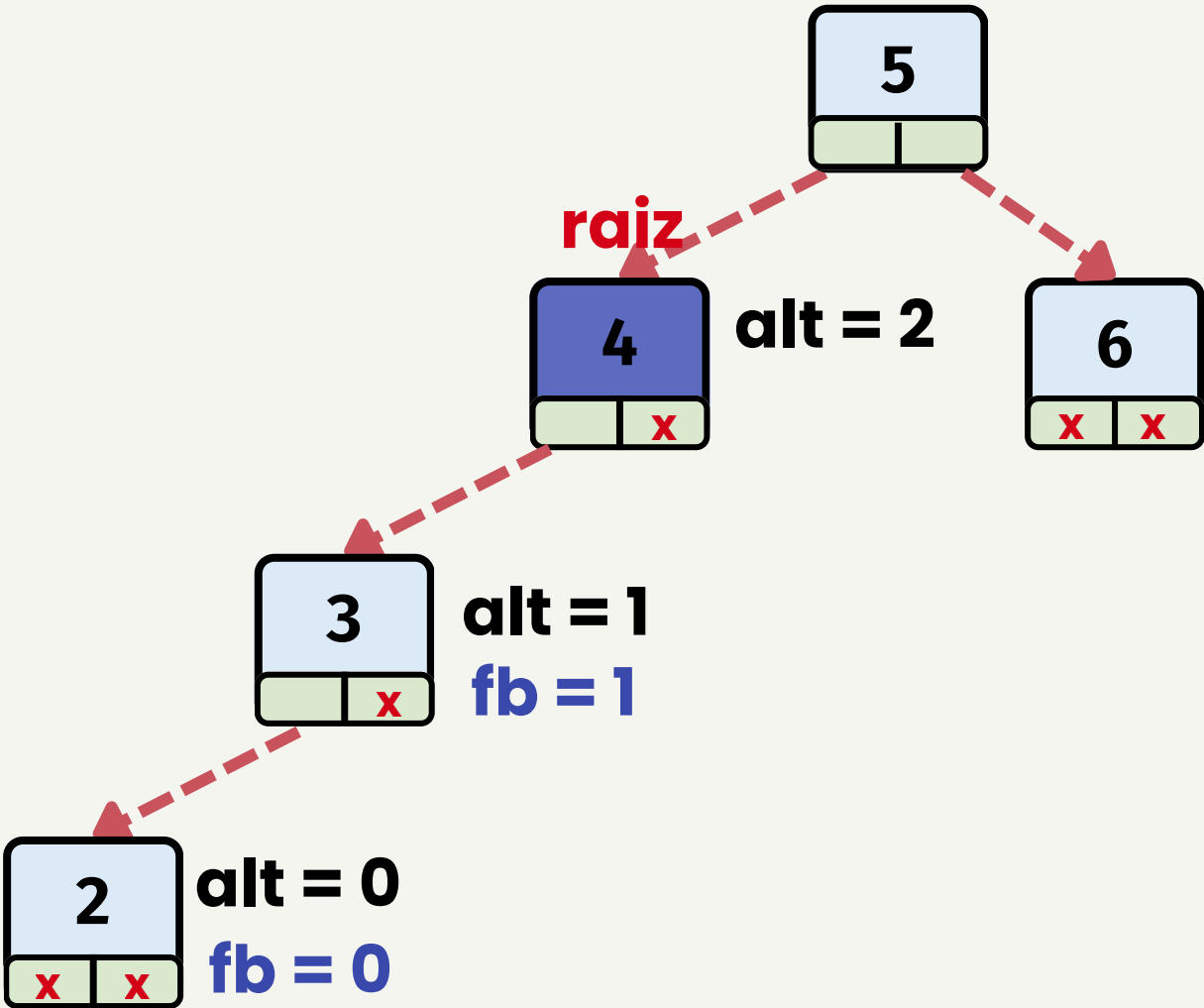
FUNÇÃO INSERIR COM RECURSÃO

INSERIR : 2

```
NoArv* inserirRecBalanceada(NoArv* raiz, int valor) {
    if (raiz == NULL) {
        return criarNo(valor);
    }
    if (valor < raiz->valor) {
        raiz->esq = inserirRec(raiz->esq, valor);
    } else if (valor > raiz->valor) {
        raiz->dir = inserirRec(raiz->dir, valor);
    }
    else {
        printf("Valor %d já existe na árvore. \n", valor);
    }

    raiz->altura = 1 + maiorValor(alturaDoNo(raiz->esq), alturaDoNo(raiz->dir));
    raiz = balancear(raiz);

    return raiz;
}
```



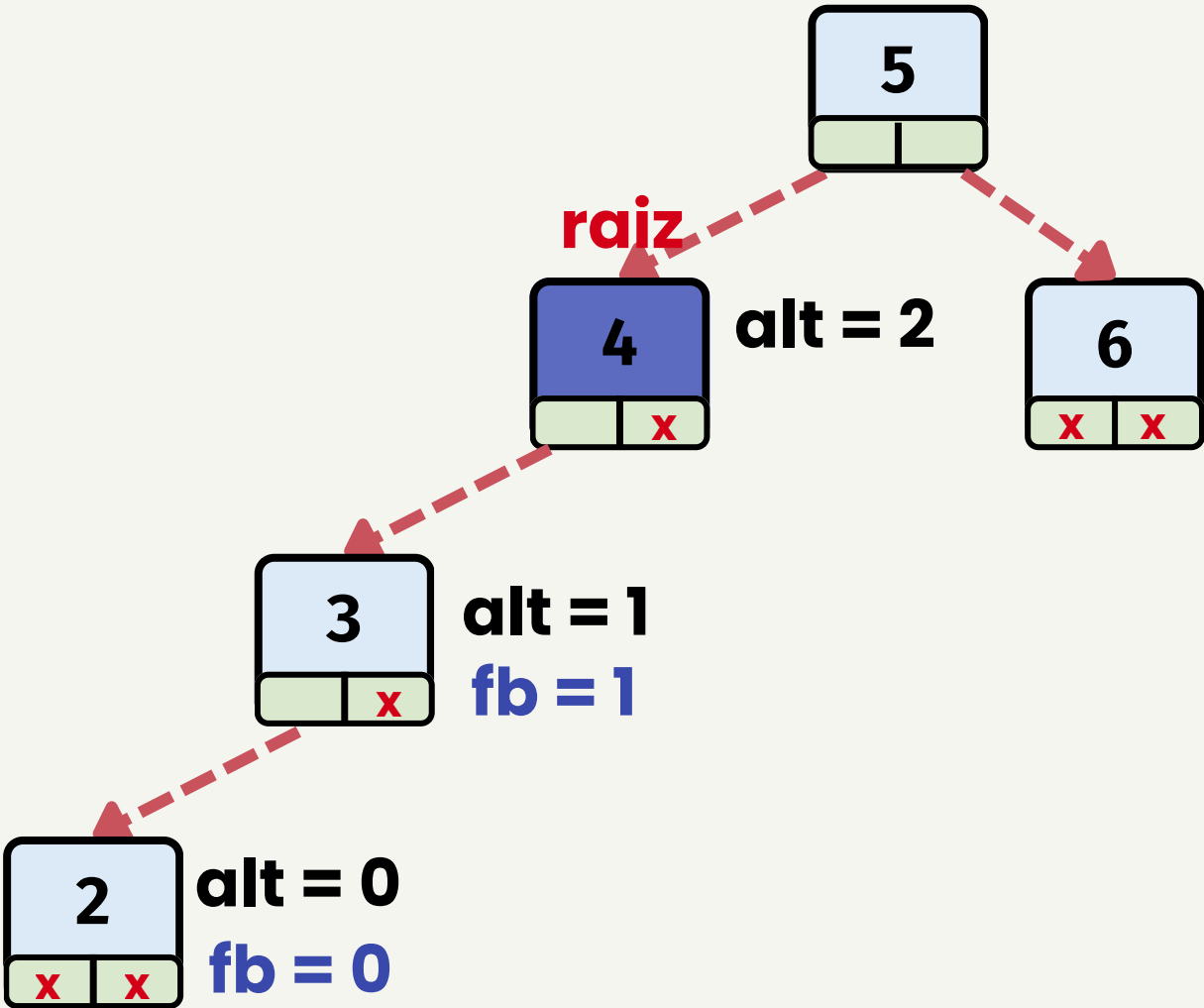
FUNÇÃO INSERIR COM RECURSÃO

INSERIR : 2

```
NoArv* inserirRecBalanceada(NoArv* raiz, int valor) {
    if (raiz == NULL) {
        return criarNo(valor);
    }
    if (valor < raiz->valor) {
        raiz->esq = inserirRec(raiz->esq, valor);
    } else if (valor > raiz->valor) {
        raiz->dir = inserirRec(raiz->dir, valor);
    }
    else {
        printf("Valor %d já existe na árvore. \n", valor);
    }

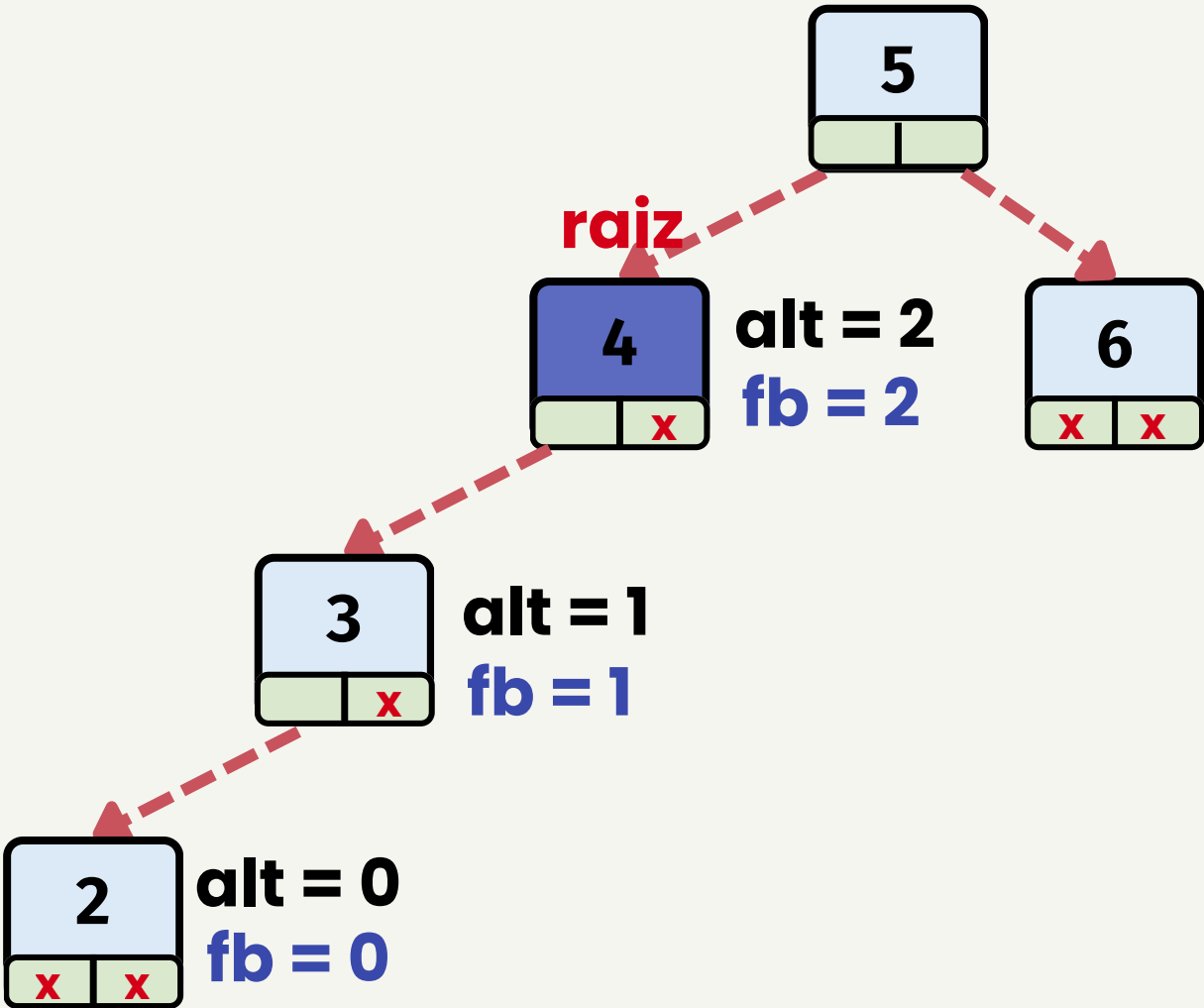
    raiz->altura = 1 + maiorValor(alturaDoNo(raiz->esq), alturaDoNo(raiz->dir));
    raiz = balancear(raiz);

    return raiz;
}
```



FUNÇÃO INSERIR COM RECURSÃO

INSERIR : 2



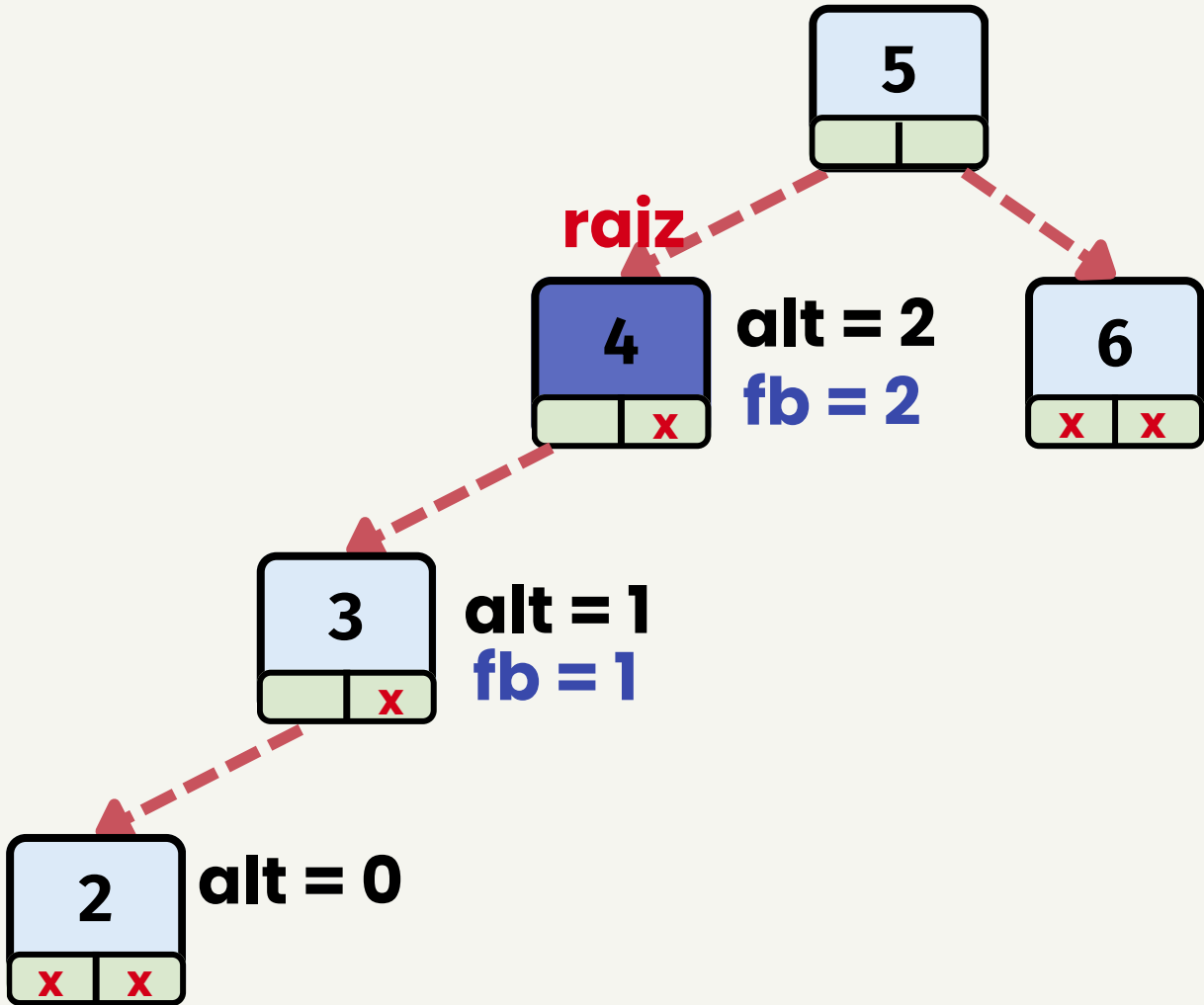
```
NoArv* balancear(NoArv* raiz) {
    int fb = fatorBalanceamento(raiz);

    if (fb > 1) {
        if (fatorBalanceamento(raiz->esq) >= 0) {
            return rotacaoDireita(raiz);
        } else {
            return rotacaoEsquerdaDireita(raiz);
        }
    } else if (fb < -1) {
        if (fatorBalanceamento(raiz->dir) <= 0) {
            return rotacaoEsquerda(raiz);
        } else {
            return rotacaoDireitaEsquerda(raiz);
        }
    }

    return raiz;
}
```

FUNÇÃO INSERIR COM RECURSÃO

INSERIR : 2



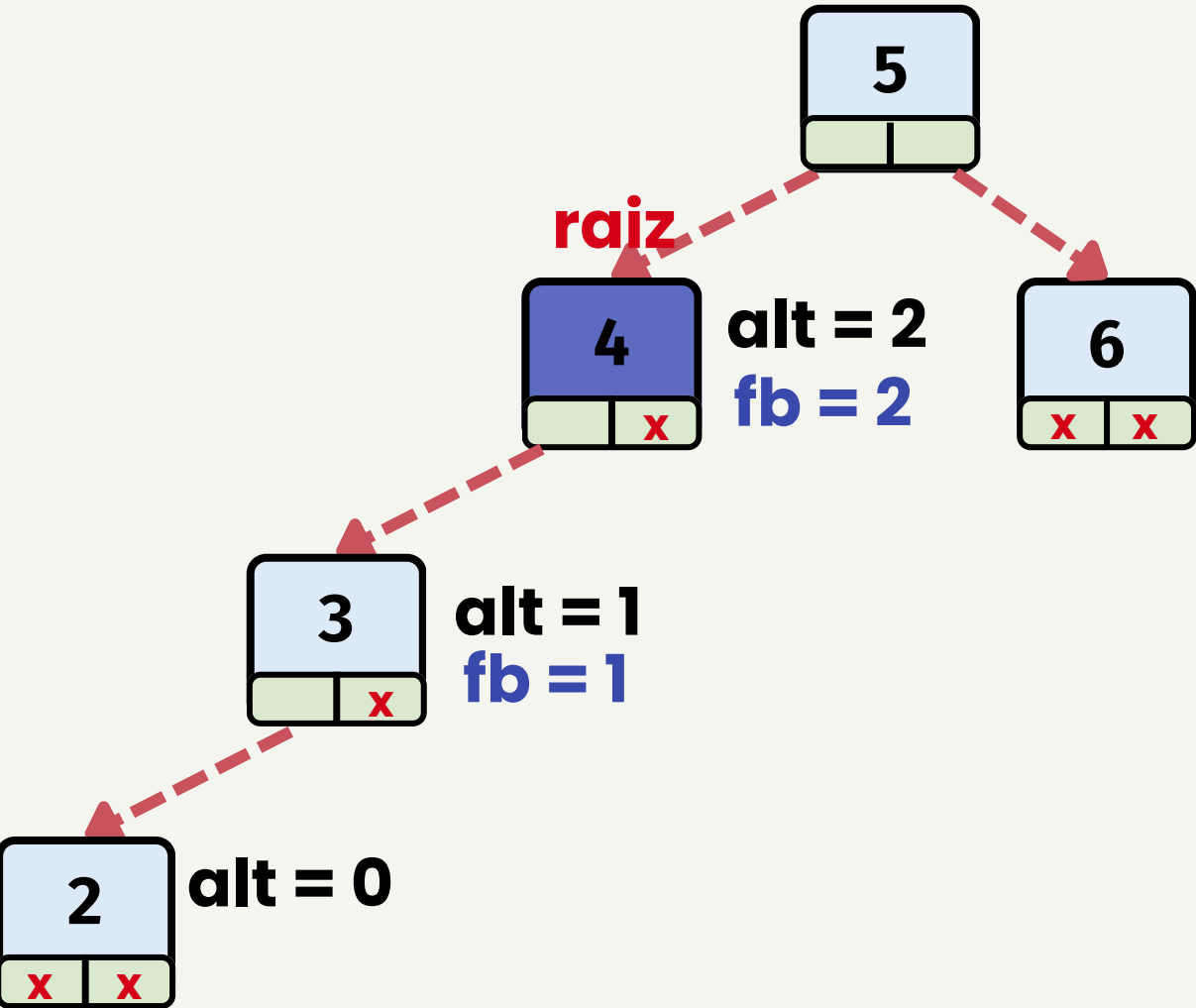
```
NoArv* balancear(NoArv* raiz) {
    int fb = fatorBalanceamento(raiz);

    if (fb > 1) {
        if (fatorBalanceamento(raiz->esq) >= 0) {
            return rotacaoDireita(raiz);
        } else {
            return rotacaoEsquerdaDireita(raiz);
        }
    } else if (fb < -1) {
        if (fatorBalanceamento(raiz->dir) <= 0) {
            return rotacaoEsquerda(raiz);
        } else {
            return rotacaoDireitaEsquerda(raiz);
        }
    }

    return raiz;
}
```

FUNÇÃO INSERIR COM RECURSÃO

INSERIR : 2



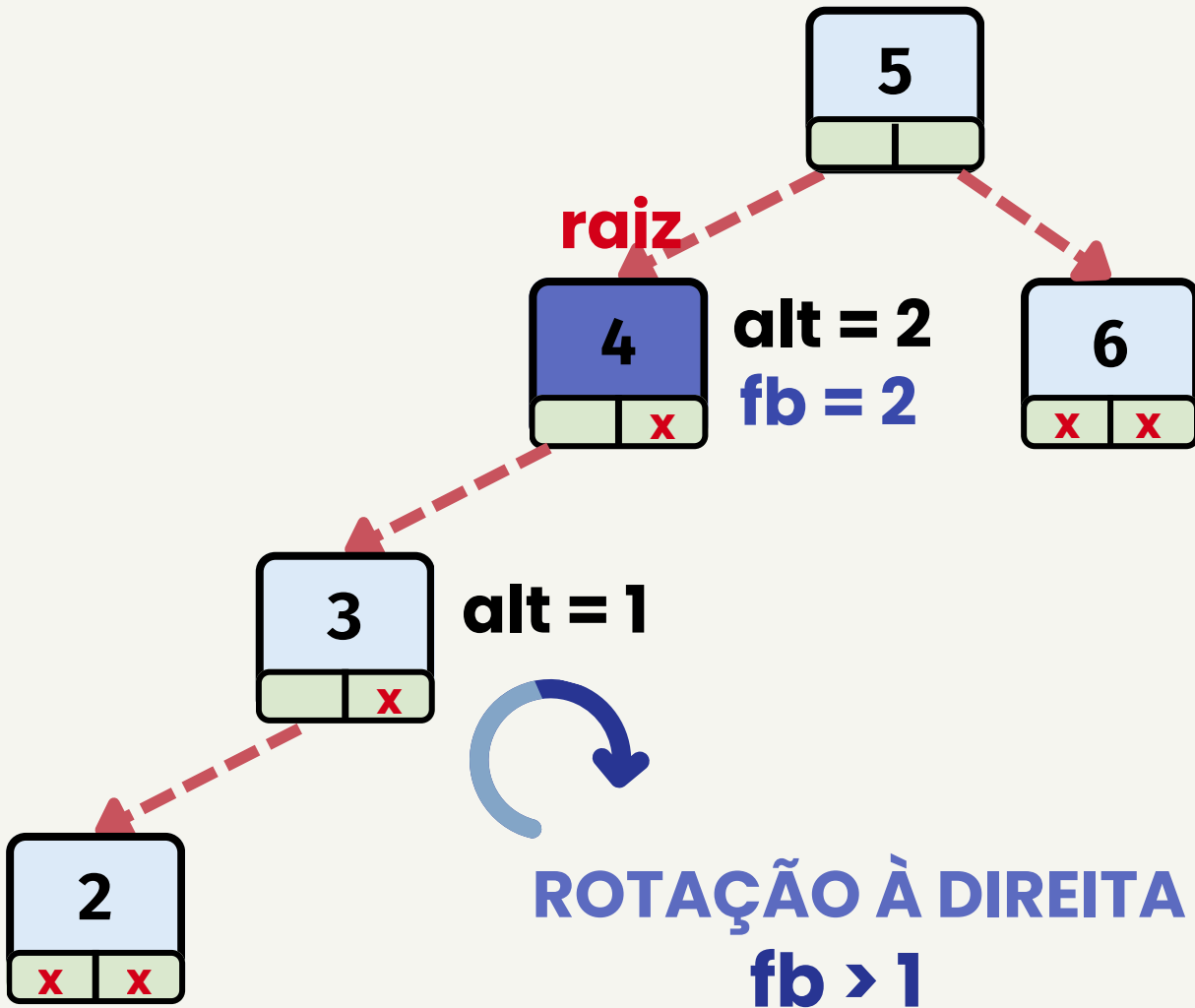
```
NoArv* balancear(NoArv* raiz) {
    int fb = fatorBalanceamento(raiz);

    if (fb > 1) {
        if (fatorBalanceamento(raiz->esq) >= 0) {
            return rotacaoDireita(raiz);
        } else {
            return rotacaoEsquerdaDireita(raiz);
        }
    } else if (fb < -1) {
        if (fatorBalanceamento(raiz->dir) <= 0) {
            return rotacaoEsquerda(raiz);
        } else {
            return rotacaoDireitaEsquerda(raiz);
        }
    }

    return raiz;
}
```

FUNÇÃO INSERIR COM RECURSÃO

INSERIR : 2



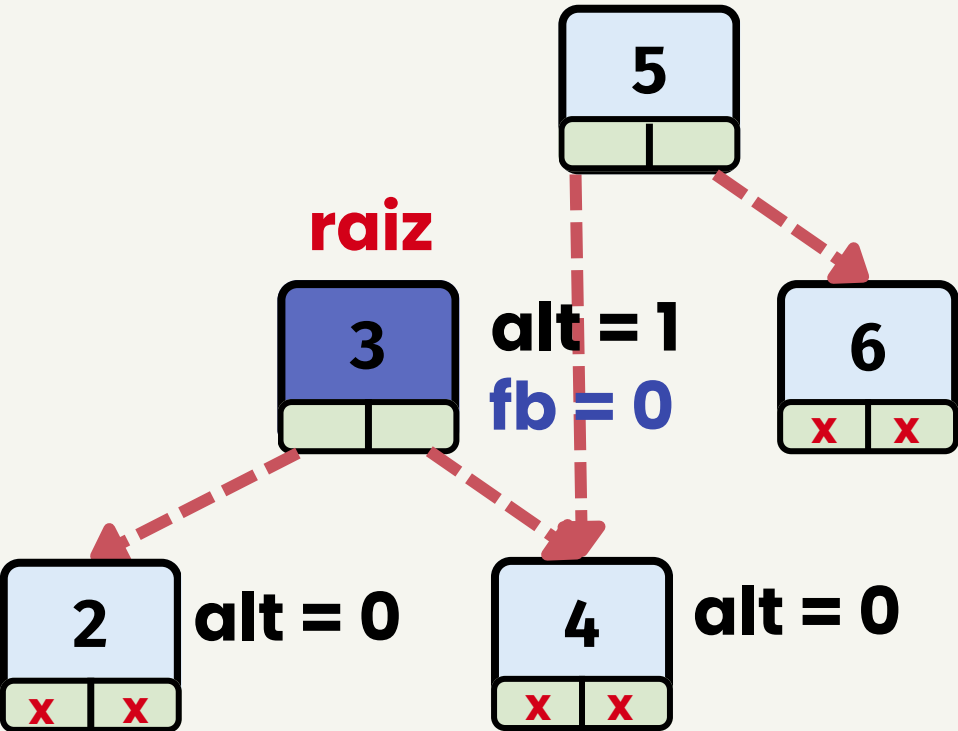
```
NoArv* balancear(NoArv* raiz) {
    int fb = fatorBalanceamento(raiz);

    if (fb > 1) {
        if (fatorBalanceamento(raiz->esq) >= 0) {
            return rotacaoDireita(raiz);
        } else {
            return rotacaoEsquerdaDireita(raiz);
        }
    } else if (fb < -1) {
        if (fatorBalanceamento(raiz->dir) <= 0) {
            return rotacaoEsquerda(raiz);
        } else {
            return rotacaoDireitaEsquerda(raiz);
        }
    }

    return raiz;
}
```

FUNÇÃO INSERIR COM RECURSÃO

INSERIR : 2



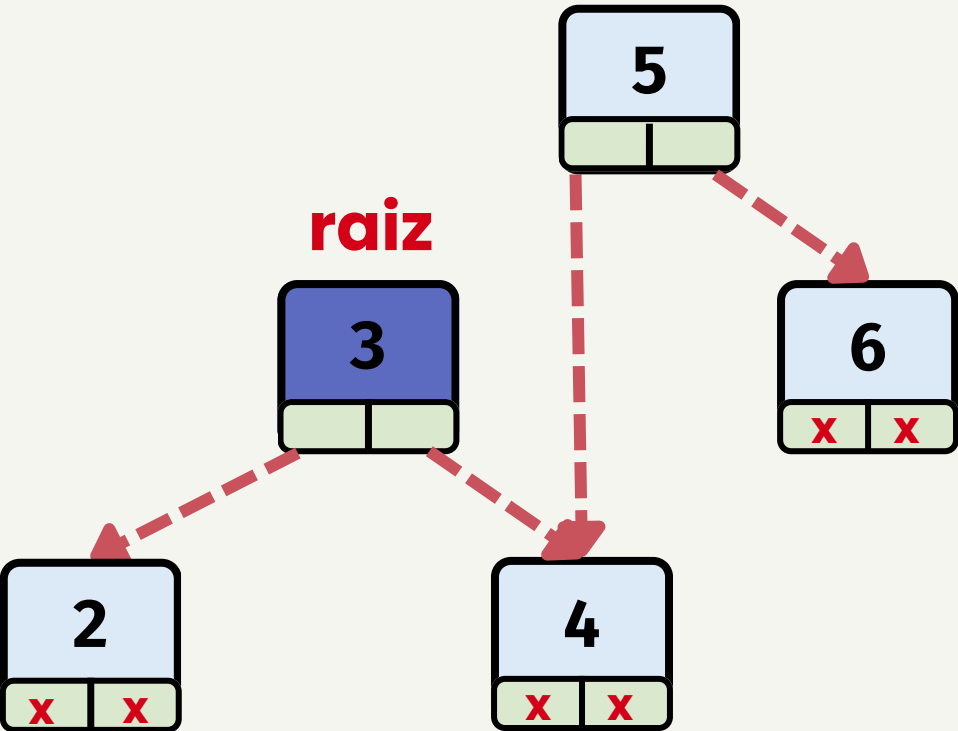
```
NoArv* balancear(NoArv* raiz) {
    int fb = fatorBalanceamento(raiz);

    if (fb > 1) {
        if (fatorBalanceamento(raiz->esq) >= 0) {
            return rotacaoDireita(raiz);
        } else {
            return rotacaoEsquerdaDireita(raiz);
        }
    } else if (fb < -1) {
        if (fatorBalanceamento(raiz->dir) <= 0) {
            return rotacaoEsquerda(raiz);
        } else {
            return rotacaoDireitaEsquerda(raiz);
        }
    }

    return raiz;
}
```

FUNÇÃO INSERIR COM RECURSÃO

INSERIR : 2



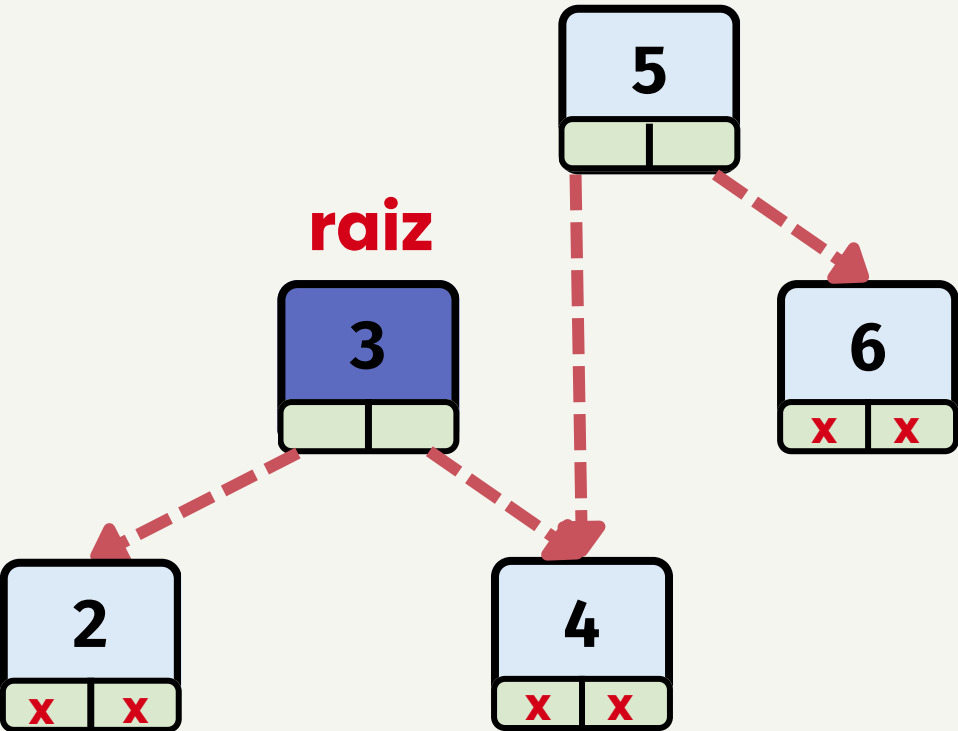
```
NoArv* inserirRecBalanceada(NoArv* raiz, int valor) {
    if (raiz == NULL) {
        return criarNo(valor);
    }
    if (valor < raiz->valor) {
        raiz->esq = inserirRec(raiz->esq, valor);
    } else if (valor > raiz->valor) {
        raiz->dir = inserirRec(raiz->dir, valor);
    }
    else {
        printf("Valor %d já existe na árvore. \n", valor);
    }

    raiz->altura = 1 + maiorValor(alturaDoNo(raiz->esq), alturaDoNo(raiz->dir));
    raiz = balancear(raiz);

    return raiz;
}
```

FUNÇÃO INSERIR COM RECURSÃO

INSERIR : 2



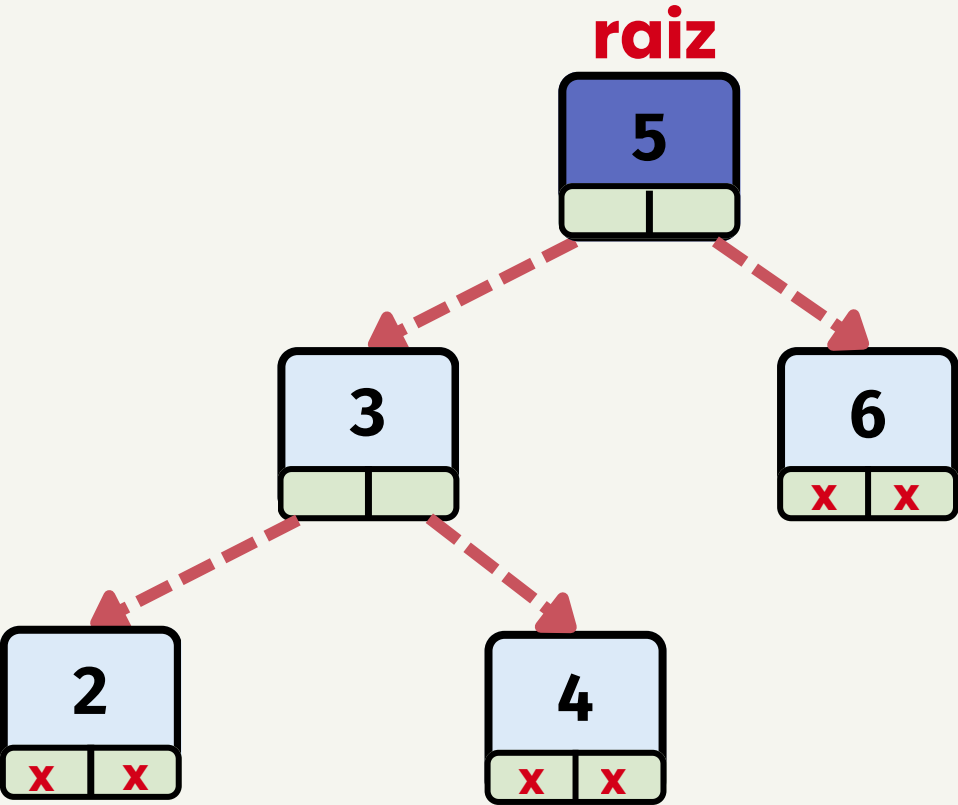
```
NoArv* inserirRecBalanceada(NoArv* raiz, int valor) {
    if (raiz == NULL) {
        return criarNo(valor);
    }
    if (valor < raiz->valor) {
        raiz->esq = inserirRec(raiz->esq, valor);
    } else if (valor > raiz->valor) {
        raiz->dir = inserirRec(raiz->dir, valor);
    }
    else {
        printf("Valor %d já existe na árvore. \n", valor);
    }

    raiz->altura = 1 + maiorValor(alturaDoNo(raiz->esq), alturaDoNo(raiz->dir));
    raiz = balancear(raiz);

    return raiz;
}
```

FUNÇÃO INSERIR COM RECURSÃO

INSERIR : 2



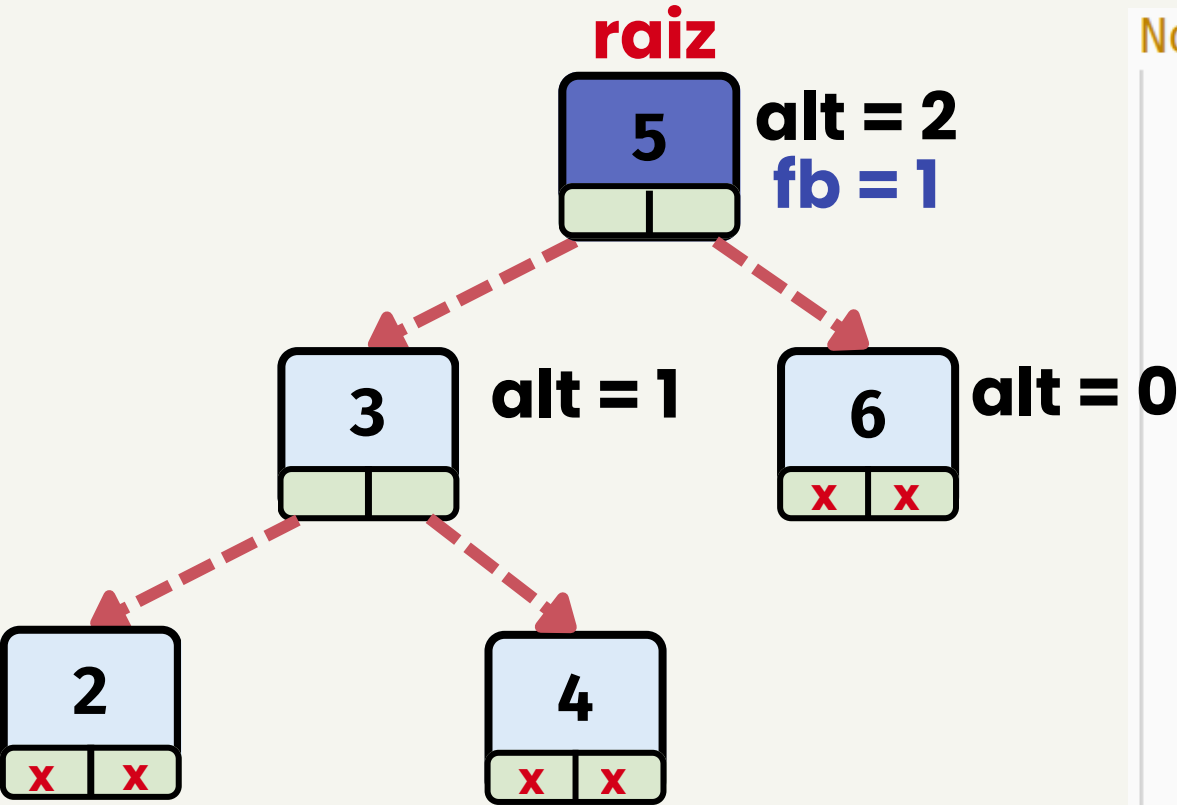
```
NoArv* inserirRecBalanceada(NoArv* raiz, int valor) {
    if (raiz == NULL) {
        return criarNo(valor);
    }
    if (valor < raiz->valor) {
        raiz->esq = inserirRec(raiz->esq, valor);
    } else if (valor > raiz->valor) {
        raiz->dir = inserirRec(raiz->dir, valor);
    }
    else {
        printf("Valor %d já existe na árvore. \n", valor);
    }

    raiz->altura = 1 + maiorValor(alturaDoNo(raiz->esq), alturaDoNo(raiz->dir));
    raiz = balancear(raiz);

    return raiz;
}
```

FUNÇÃO INSERIR COM RECURSÃO

INSERIR : 2



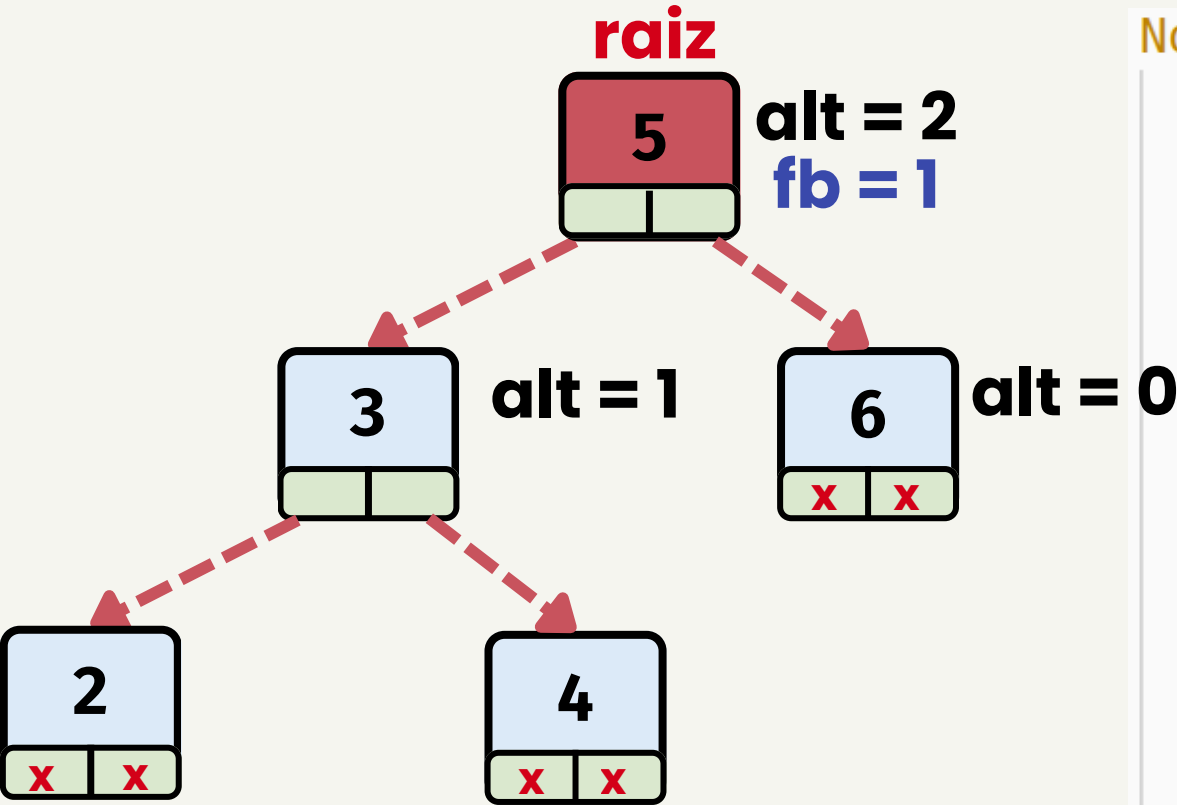
```
NoArv* inserirRecBalanceada(NoArv* raiz, int valor) {
    if (raiz == NULL) {
        return criarNo(valor);
    }
    if (valor < raiz->valor) {
        raiz->esq = inserirRec(raiz->esq, valor);
    } else if (valor > raiz->valor) {
        raiz->dir = inserirRec(raiz->dir, valor);
    }
    else {
        printf("Valor %d já existe na árvore. \n", valor);
    }

    raiz->altura = 1 + maiorValor(alturaDoNo(raiz->esq), alturaDoNo(raiz->dir));
    raiz = balancear(raiz);

    return raiz;
}
```

FUNÇÃO INSERIR COM RECURSÃO

INSERIR : 2



```
NoArv* inserirRecBalanceada(NoArv* raiz, int valor) {
    if (raiz == NULL) {
        return criarNo(valor);
    }
    if (valor < raiz->valor) {
        raiz->esq = inserirRec(raiz->esq, valor);
    } else if (valor > raiz->valor) {
        raiz->dir = inserirRec(raiz->dir, valor);
    }
    else {
        printf("Valor %d já existe na árvore. \n", valor);
    }

    raiz->altura = 1 + maiorValor(alturaDoNo(raiz->esq), alturaDoNo(raiz->dir));
    raiz = balancear(raiz);

    return raiz;
}
```

FUNÇÃO REMOVER COM RECURSÃO

```
NoArv* removerRecBalanceada(NoArv* raiz, int valor) {
    if (raiz == NULL) { // Caso base: valor não encontrado
        return NULL;
    }

    if (valor < raiz->valor) { // Buscar na subárvore esquerda
        raiz->esq = removerRecBalanceada(raiz->esq, valor);
    }
    else if (valor > raiz->valor) { // Buscar na subárvore direita
        raiz->dir = removerRecBalanceada(raiz->dir, valor);
    }
    else { // Nó encontrado, realizar a remoção
        // Caso 1: Nó com apenas um filho ou nenhum filho
        if (raiz->esq == NULL || raiz->dir == NULL) {
            NoArv* temp = (raiz->esq != NULL) ? raiz->esq : raiz->dir;
            free(raiz);
            return temp; // Retorna o filho (ou NULL se não houver filhos)
        }

        // Caso 2: Nó com dois filhos
        NoArv* sucessor = raiz->dir;
        while (sucessor->esq != NULL) {
            sucessor = sucessor->esq;
        }
        raiz->valor = sucessor->valor; // Substituir o valor do nó pelo sucessor
        raiz->dir = removerRecBalanceada(raiz->dir, sucessor->valor); // Remover o sucessor
    }

    raiz->altura = 1 + maiorValor(alturaDoNo(raiz->esq), alturaDoNo(raiz->dir));
    raiz = balancear(raiz); // Balancear a árvore após a remoção

    return raiz;
}
```

FUNÇÃO REMOVER COM RECURSÃO

```

NoArv* removerRecBalanceada(NoArv* raiz, int valor) {
    if (raiz == NULL) { // Caso base: valor não encontrado
        return NULL;
    }

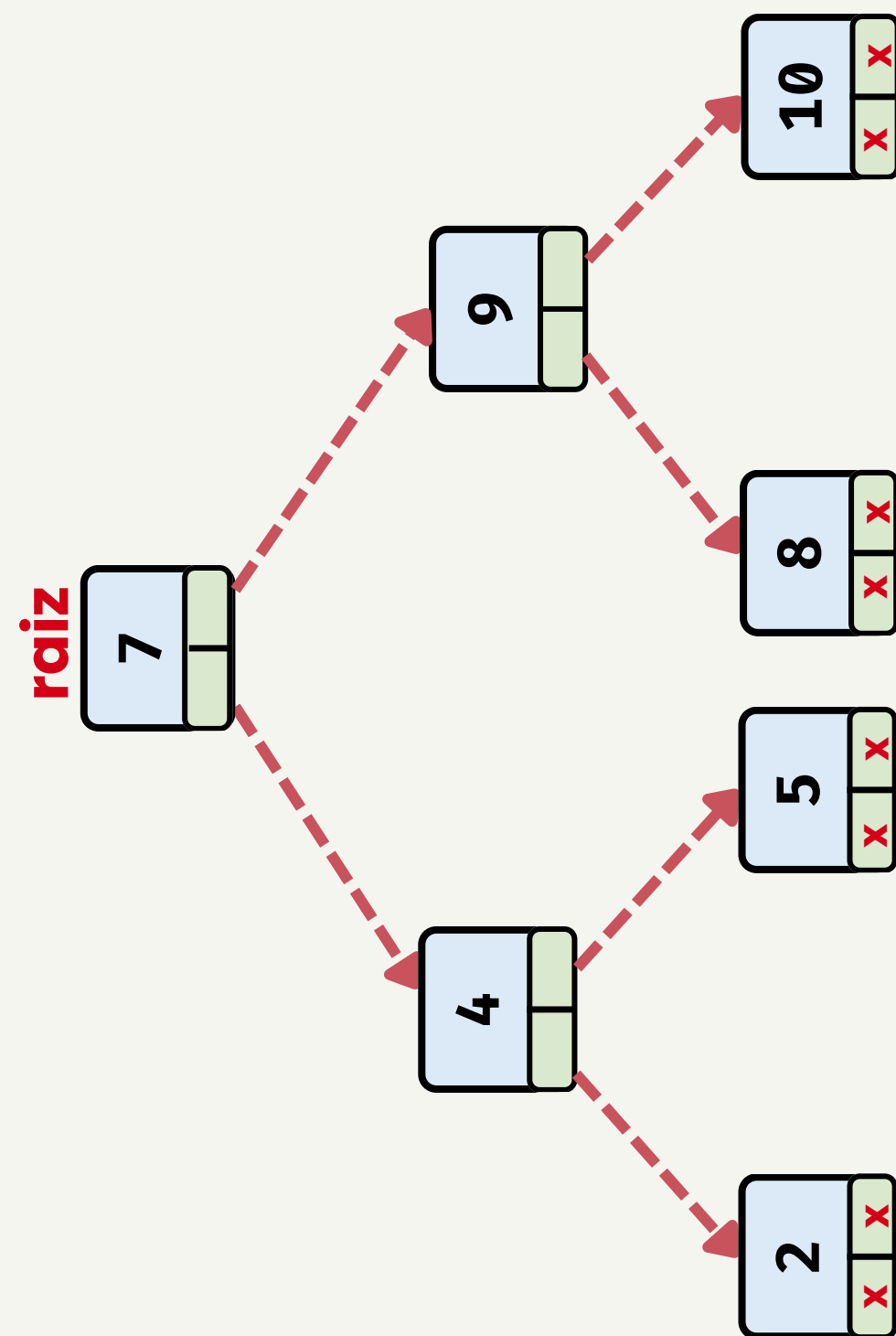
    if (valor < raiz->valor) { // Buscar na subárvore esquerda
        raiz->esq = removerRecBalanceada(raiz->esq, valor);
    }
    else if (valor > raiz->valor) { // Buscar na subárvore direita
        raiz->dir = removerRecBalanceada(raiz->dir, valor);
    }
    else { // Nó encontrado, realizar a remoção
        // Caso 1: Nó com apenas um filho ou nenhum filho
        if (raiz->esq == NULL || raiz->dir == NULL) {
            NoArv* temp = (raiz->esq != NULL) ? raiz->esq : raiz->dir;
            free(raiz);
            return temp; // Retorna o filho (ou NULL se não houver filhos)
        }

        // Caso 2: Nó com dois filhos
        NoArv* sucessor = raiz->dir;
        while (sucessor->esq != NULL) {
            sucessor = sucessor->esq;
        }
        raiz->valor = sucessor->valor; // Substituir o valor do nó pelo sucessor
        raiz->dir = removerRecBalanceada(raiz->dir, sucessor->valor); // Remover o sucessor
    }

    raiz->altura = 1 + maiorValor(alturaDoNo(raiz->esq), alturaDoNo(raiz->dir));
    raiz = balancear(raiz); // Balancear a árvore após a remoção

    return raiz;
}
    
```

IMPRIMIR



```
void imprimir(NoArv* raiz, int nivel) {
    int i;
    if(raiz){
        imprimir(raiz->dir, nivel + 1);
        printf("\n\n");

        for(i = 0; i < nivel; i++) {
            printf("\t");
        }
        printf("%d", raiz->valor);
        imprimir(raiz->esq, nivel + 1);
    }
}
```

VISUALIZAÇÃO / TESTES

<https://www.cs.usfca.edu/~galles/visualization/AVLtree.html>

Ex.: Considere o trecho abaixo:

```

int alturaDoNo(NoArv* no) {
    if (no == NULL) {
        return -1;
    }
    return no->altura;
}

int fatorBalanceamento(NoArv* no) {
    if (no == NULL) {
        return 0;
    }
    return alturaDoNo(no->esq) - alturaDoNo(no->dir);
}

NoArv* balancear(NoArv* raiz) {
    int fb = fatorBalanceamento(raiz);

    if (fb > 1) {
        if (fatorBalanceamento(raiz->esq) >= 0) {
            return rotacaoDireita(raiz);
        } else {
            return rotacaoEsquerdaDireita(raiz);
        }
    } else if (fb < -1) {
        if (fatorBalanceamento(raiz->dir) <= 0) {
            return rotacaoEsquerda(raiz);
        } else {
            return rotacaoDireitaEsquerda(raiz);
        }
    }

    return raiz;
}

```

EXERCÍCIOS

Agora considere que os valores: **30, 10, 20** foram inseridos, nessa ordem, em uma árvore AVL.

Responda:

- a) Desenhe a **árvore resultante** com a inserção dos valores(sem balancear).
- b) Qual é o **fator de balanceamento** do nó raiz?
- c) Ao entrar na função **balancear**, qual condição será **satisfeita e qual rotação será aplicada?**
- e) Desenhe a árvore final **após** o balanceamento.

EXERCÍCIOS

**Ex1.:Faça a construção de uma árvore AVL que insira os valores:
{76, 60, 15, 65, 8, 46, 55, 22,66,72,58}**

EXERCÍCIOS

Ex1.:Faça a construção de uma árvore AVL que insira os valores:
{76, 60, 15, 65, 8, 46, 55, 22,66,72,58}

Agora remova o nó 66

EXERCÍCIOS

**Ex1.:Faça a construção de uma árvore AVL que insira os valores:
{76, 60, 15, 65, 8, 46, 55, 22,66,72,58}**

Agora remova o nó 66

Agora remova os nós 55 e 58, nesta ordem.

EXERCÍCIOS

Ex2.:

Implemente a função recursiva:

```
int contarNos(NoArv* raiz)
```

Ela deve retornar a quantidade total de nós da árvore.

EXERCÍCIOS

Ex2.:

Implemente a função recursiva:

```
int contarNos(NoArv* raiz)
```

Ela deve retornar a quantidade total de nós da árvore.

```
int contarNos(NoArv* raiz) {  
    if (raiz == NULL) {// Caso base: árvore vazia  
        return 0;  
    }  
    return 1 + contarNos(raiz->esq) + contarNos(raiz->dir);  
}
```

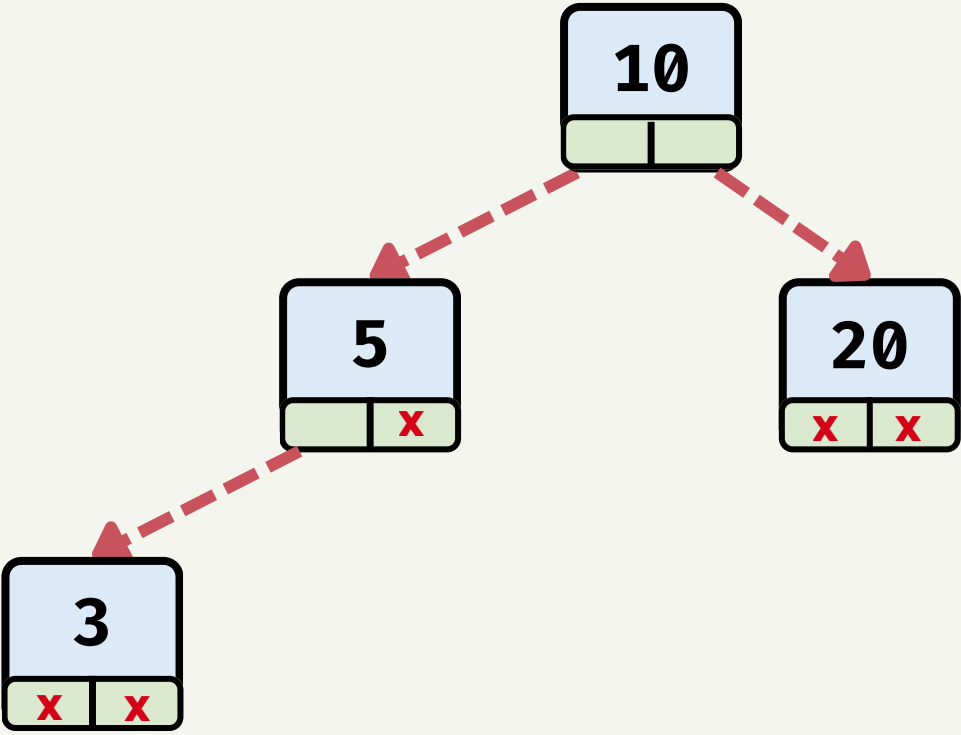
EXERCÍCIOS

Ex2.:
Implemente a função recursiva:

```
int contarNos(NoArv* raiz)
```

**Ela deve retornar a quantidade
total de nós da árvore.**

```
int contarNos(NoArv* raiz) {
    if (raiz == NULL) {// Caso base: árvore vazia
        return 0;
    }
    return 1 + contarNos(raiz->esq) + contarNos(raiz->dir);
}
```



EXERCÍCIOS

Ex3.: Implemente a função:

```
void verificarBalanceamentoRaiz(NoArv* raiz)
```

Ela deve calcular o fator de balanceamento da raiz e imprimir se ela está balanceada ou desbalanceada.

Utilize a função existente:

```
int fatorBalanceamento(NoArv* no) {  
    if (no == NULL) {  
        return 0;  
    }  
    return alturaDoNo(no->esq) - alturaDoNo(no->dir);  
}
```

EXERCÍCIOS

Ex3.: Implemente a função:

```
void verificarBalanceamentoRaiz(NoArv* raiz)
```

Ela deve calcular o fator de balanceamento da raiz e imprimir se ela está balanceada ou desbalanceada.

Utilize a função existente:

```
int fatorBalanceamento(NoArv* no) {  
    if (no == NULL) {  
        return 0;  
    }  
    return alturaDoNo(no->esq) - alturaDoNo(no->dir);  
}
```

Desbalanceada: $fb > 1$ ou $fb < -1$

EXERCÍCIOS

Ex3.: Implemente a função:

```
void verificarBalanceamentoRaiz(NoArv* raiz)
```

Ela deve calcular o fator de balanceamento da raiz e imprimir se ela está balanceada ou desbalanceada.

```
void verificarBalanceamentoRaiz(NoArv* raiz) {  
    if (raiz == NULL) {  
        printf("A árvore está vazia.\n");  
        return;  
    }  
}
```

EXERCÍCIOS

Ex3.: Implemente a função:

```
void verificarBalanceamentoRaiz(NoArv* raiz)
```

Ela deve calcular o fator de balanceamento da raiz e imprimir se ela está balanceada ou desbalanceada.

```
void verificarBalanceamentoRaiz(NoArv* raiz) {  
    if (raiz == NULL) {  
        printf("A árvore está vazia.\n");  
        return;  
    }  
  
    int fb = fatorBalanceamento(raiz);
```

EXERCÍCIOS

Ex3.: Implemente a função:

```
void verificarBalanceamentoRaiz(NoArv* raiz)
```

Ela deve calcular o fator de balanceamento da raiz e imprimir se ela está balanceada ou desbalanceada.

```
void verificarBalanceamentoRaiz(NoArv* raiz) {  
    if (raiz == NULL) {  
        printf("A árvore está vazia.\n");  
        return;  
    }  
  
    int fb = fatorBalanceamento(raiz);  
  
    printf("Raiz: %d\n", raiz->valor);  
    printf("Fator de balanceamento da raiz: %d\n", fb);  
}
```

EXERCÍCIOS

Ex3.: Implemente a função:

```
void verificarBalanceamentoRaiz(NoArv* raiz)
```

Ela deve calcular o fator de balanceamento da raiz e imprimir se ela está balanceada ou desbalanceada.

```
void verificarBalanceamentoRaiz(NoArv* raiz) {  
    if (raiz == NULL) {  
        printf("A árvore está vazia.\n");  
        return;  
    }  
  
    int fb = fatorBalanceamento(raiz);  
  
    printf("Raiz: %d\n", raiz->valor);  
    printf("Fator de balanceamento da raiz: %d\n", fb);  
  
    if (fb >= -1 && fb <= 1) {  
        printf("A raiz está balanceada.\n");  
    } else {  
        printf("A raiz está desbalanceada.\n");  
    }  
}
```